BESCHREIBUNG

Bausteininterpretationssystem

Bausteine, Kleines Eingabeprogramm (KEP) und Minimalprogramm

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205-Interpreter-BIS.md	0.03-028
Stand	10.11.2026	
Umfang	26 Seiten, A4 hoch	
Erzeugt mit	erzeuge_bis_pdf.py	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

Inhaltsverzeichnis

BIS-1 Allgemeine Beschreibung des BIS . . . 3
BIS-1.1 Zahlendarstellungen des BIS . . . 3
BIS-1.2 Anschlusszellen: wie ein Programm die Bausteine findet . . . 4
BIS-2 Beschreibung der Bausteine . . . 5
BIS-2.1 Das kleine Eingabeprogramm (KEP): Voraussetzung des BIS . . . 5
BIS-2.2 Das Minimalprogramm . . . 6
BIS-3 Abgrenzung zu Interpreter und Assembler . . . 20
BIS-A Anhang: Ansprungzellen, Anschlußbedingungen und Arbeitszellen . . . 20
BIS-A.1 Eingabe- und Hilfsprogramme (Anschlußzellen 201 bis 430) . . . 20
BIS-A.2 Rechen-, Ausgabe- und Satzeingabeprogramme (Ansprungzellen 7540 bis 7575) . . . 22
BIS-A.3 Arbeitszellen des Minimalprogramms . . . 23
BIS-A.4 Funktionen des Minimalprogramms v6 (Stand 24.10.2026) . . . 26

C 8205 – Bausteininterpretationssystem

Quelle: C8205-Interpreter-BIS.md · im Wortlaut der Quelle

Version: 0.03-028

Dieses Dokument gibt den Inhalt der Quelle wieder und ordnet ihn in das Projekt ein. Die Gliederungsnummern der Quelle werden nicht übernommen; Verweise auf Abschnitte oder andere Dokumente enthält dieses Dokument nicht. Zusätze dieses Dokuments, die nicht im Wortlaut der Quelle stehen, sind als „Anmerkung" gekennzeichnet. Unsichere Lesestellen und Widersprüche zwischen Quellen sind am Ende des Dokuments gesammelt (derzeit keine; eine Randnotiz ist in BIS-1.1 als „nicht übernommen“ vermerkt).

Einordnung im Projekt: Das BIS ist das dritte Teilprojekt neben Interpreter und Assembler.

BIS-1 Allgemeine Beschreibung des BIS

- **Wesen:** Das vollständige Bausteininterpretiersystem „BIS" stellt ein **offenes Unterprogrammsystem** dar. Es enthält alle Routinen für
  - Ein- und Ausgabe von Dezimalzahlen,
  - Multiplikation und Division von Festkomma- und Gleitkommazahlen,
  - Addition und Subtraktion von Gleitkommazahlen,
  - eine Reihe von Serviceprogrammen.
- **Zeitoptimal:** Alle Bausteine sind so aufgebaut, dass sie zeitoptimal arbeiten.
- **Länge:** Ein Baustein umfasst zwischen 67_8 und 377_8 Speicherplätzen.
- **Rechengeschwindigkeit:** Bei der für Kleinrechner relativ hohen Rechengeschwindigkeit ist der Zeitaufwand für die Abarbeitung der Bausteine, z. B. gegenüber der Ein- bzw. Ausgabe, gering.
- **Zusammenstellung und Ablage:** Das BIS ist so konzipiert, dass mit einigen Einschränkungen die einzelnen Bausteine nach Bedarf zusammengestellt und auf beliebigen Bereichen der Magnettrommel abgespeichert werden können. Dabei ist gewährleistet, dass alle Bausteine trotz frei wählbarer Abspeicherung unter **festen Adressen** erreichbar sind. Das erleichtert dem Programmierer die Arbeit: Er braucht die Abspeicheradressen nicht zu kennen.
- **Herausgeber:** Das BIS wird vom VEB Kombinat Zentronik zur Verfügung gestellt.
- **Gruppen:** Das BIS besteht aus einer Reihe von Bausteinen, die in vier Gruppen eingeteilt werden können: Eingabebausteine, Rechenbausteine, Ausgabebausteine, Serviceprogramme.
- **Zahlenarten:** Das System gestattet, sowohl mit Festkommazahlen als auch mit Gleitkommazahlen zu arbeiten.
- **Programmierung:** Die praktische Programmierung erfolgt in der Maschinensprache.
- **Einsatz:** Es eignet sich grundsätzlich für die Bearbeitung wissenschaftlich-technischer, mathematischer und mit Einschränkungen auch ökonomischer Probleme.
- **Interpretation der Zahlen:** Sowohl die Ein- und Ausgabe von Zahlen als auch die Interpretation des Inhalts der Maschinenworte werden durch das jeweils zur Anwendung kommende Interpretiersystem bestimmt, das die entsprechenden Routinen enthalten muss.

Anmerkung (Einordnung): Die Bausteine sind die Vorläufer dessen, was spätere Rechner als Betriebssystem bereitstellen. Der Rechner C 8205 selbst kennt nur die fest verdrahtete Eingabe eines Zeichens; alles Weitere (Worte einlesen, Dezimalzahlen wandeln, Multiplikation, Division, Gleitkomma, Ausgabe) leisten Programme aus dem BIS. Die Gleitkomma- und die Tetradendarstellung sind daher Konventionen des BIS.

BIS-1.1 Zahlendarstellungen des BIS

Anmerkung (Einordnung): Intern verwendet das BIS ganze Festkommazahlen (Komma hinter dem letzten Bit) und Gleitkommazahlen zur Basis 2 mit normalisierter Mantisse und um 32 erhöhtem Exponenten. Diese Deutungen des Maschinenwortes beschreibt dieser Abschnitt nicht; er beschreibt, was erst das BIS hinzufügt: die externe Darstellung der Gleitkommazahlen und die Tetradendarstellung. Die Zahlenbereiche der Eingabe (BIS-2.2.6) sind enger als der Zahlenbereich des Maschinenwortes; so gilt bei $K_1 = 5$ die Grenze $2^{31} - 1$ für die softwareseitige Festkommaarithmetik des BIS, nicht für das Maschinenwort.

Gleitkommazahlen, externe Darstellung (Quelle: BIS-Dokumentation, Ergänzung 07.10.2026)

- Die Quelle beschreibt das Programmsystem BIS, nicht die Rechner-Hardware. Die externe Darstellung und der Eingabezahlenbereich gelten daher für die Ein- und Ausgabe unter BIS.
- Die **externe** Darstellung einer Gleitkommazahl (Ein- und Ausgabe) kennt zwei Formen: die übliche Dezimalschreibweise oder die Gleitkommadarstellung, die sich für Mantisse und Exponent auf das Dezimalsystem bezieht: $z = \pm m \cdot 10^E$.

- Normalisierungsvorschrift: $0{,}1 \le m < 1$ (für $z \neq 0$). Der Exponent E wird **nicht** erhöht (anders als intern $e' = e + 32$); er besteht aus Vorzeichen und Betrag: $E = \pm |E|$.
- **Form der externen Gleitkommazahl:** Vorzeichen der Mantisse, höchstens acht dezimale Mantissenziffern m_1 m_2 … m_8, Exponentenvorzeichen und Exponent E; Schreibweise $z = (\pm)\ m_1m_2m_3m_4m_5m_6m_7m_8\ (\pm)\ E$ (die Vorzeichen in Klammern sind optional).
- **Eingabezahlenbereich:** Da E bei der Eingabe nicht größer als 8 sein darf, gilt $|z| \le (1 - 10^{-8}) \cdot 10^8$. Die kleinste eingebbare positive Zahl $\neq 0$ ist $z = 0{,}1 \cdot 10^{-8} = 10^{-9}$. Die Grenzen je Eingabekennzeichen K_1 stehen in BIS-2.2.6.
- **Dezimalschreibweise:** $z = (\pm)\ a{,}b$. Der Eingabezahlenbereich ist derselbe: $|z| \le (1 - 10^{-8}) \cdot 10^8$. Die größte eingebbare Zahl ist 99 999 999, die kleinste $\neq 0$ ist 0,000000001.
- **Genauigkeit (handschriftlicher Randvermerk der Quelle):** m = 26 Dual = 7 Dezimalstellen; nur 7 Dezimalziffern lassen sich genau verarbeiten. Die achte Mantissenziffer der externen Form ist daher nicht mehr exakt darstellbar.
- Die externe Darstellung ist wie die interne reine Konvention des Programmsystems (BIS) bzw. des Programms; der Rechner selbst kennt weder Gleitkomma noch Dezimalumwandlung.
- Anmerkung (nicht übernommen): Die handschriftliche Randnotiz zu Seite 1.3 (Zahlenbeispiele mit „72 Stellen") ist in der Kopie nicht eindeutig lesbar und passt nicht zum Zahlenbereich von $2^{31} \approx 2 \cdot 10^9$; sie gehört offenbar nicht zur Definition des C 8205.

Tetradendarstellung (Quelle: BIS-Dokumentation, Ergänzung 07.10.2026)

- **Idee:** Bei der Tetradendarstellung von Dezimalzahlen wird jede Dezimalziffer als vierstellige Dualzahl (Tetrade) geschrieben. Sie unterscheidet sich von der reinen Dezimaldarstellung nur dadurch, dass für eine Ziffer nicht ein, sondern vier Zeichen verwendet werden. Handschriftlicher Randvermerk der Quelle (am Rand abgeschnitten, sinngemäß): nur für Ein- und Ausgabe.
- **Intern:** In den 33 Dualstellen des Maschinenwortes lassen sich 8 Tetraden unterbringen. Zuordnung (höchstwertige Ziffer links):

Wortstellen	z_0–z_3	z_4–z_7	z_8–z_{11}	z_{12}–z_{15}	z_{16}–z_{19}	z_{20}–z_{23}	z_{24}–z_{27}	z_{28}–z_{31}	z_{32}
Tetrade	t_7	t_6	t_5	t_4	t_3	t_2	t_1	t_0	–

- Ein Maschinenwort fasst höchstens 8 Tetraden ($8 \cdot 4 = 32$ Bit); das 33. Bit, **z_{32}, bleibt frei**. Die Tetraden stehen damit **linksbündig** im Wort: t_7 beginnt bei z_0, die niederwertigste Tetrade t_0 endet bei z_{31}, nicht bei z_{32}. Beim Auswerten eines Tetradenworts bleibt z_{32} unberücksichtigt.
- Die Tetraden t_7 … t_0 sind nicht mit den Oktal-Triaden des Maschinenwortes (t_{10} … t_0, 3 Bit) zu verwechseln; die Bezeichnung t hat hier eine andere Bedeutung. Tetraden und Triaden liegen im Wort nicht deckungsgleich.
- **Extern:** Die Zahl besteht aus höchstens acht Dezimalziffern **ohne Vorzeichen**. Zahlenbereich: $z \le 10^8 - 1 =$ 99 999 999 (Ziffern 0 bis 9, handschriftlicher Zusatz „Dezimal 0–9").
- **Beispiel der Quelle:** externes Wort 2 8 1 3 6 9 0 5 ergibt das interne Wort `0010 1000 0001 0011 0110 1001 0000 0101 0` (33 Bit, letzte Stelle $z_{32} = 0$), oktal `12011551012` (nachgerechnet).
- **Rechnen:** Im Bausteininterpretiersystem gibt es nur Programme für die Ein- und Ausgabe von Tetraden. Rechenoperationen mit Zahlen in Tetradendarstellung können nicht durchgeführt werden. Der Rechner selbst hat dafür keine Befehle; die Darstellung ist wie die Gleitkommadarstellung reine Programmkonvention.
- Anmerkung: Der Assembler `c82ass` kennt kein Tetraden-Literal; ein Tetradenwort wird wie jedes andere Wort als Oktalzahl mit führender 0 angegeben (obiges Beispiel: `dw 012011551012`).

BIS-1.2 Anschlusszellen: wie ein Programm die Bausteine findet

Anmerkung (Festlegung des Projektinhabers, 07.11.2026; zentrales Prinzip des BIS, steht so nicht im Wortlaut der Quelle):

- **Ausgangslage:** Ein eigenständiges Programm (Standalone-Programm, einfaches Testprogramm) braucht nur seinen Assemblercode. Im Praxiseinsatz werden dagegen zuerst ein oder mehrere Programme, die Bausteine, über das Minimalprogramm geladen und zuletzt das eigentliche, zum Beispiel zu testende Programm. Jeder Baustein liefert bestimmte Programmfunktionen, die aufeinander abgestimmt sind; das zu testende Programm benötigt diese Funktionen. Die Bausteine dürfen in gewissen Grenzen frei im Speicher liegen.
- **Das Problem:** Das Programm kann nicht wissen, wo ein Baustein im Speicher liegt, und braucht doch die Einsprungstelle seiner Unterprogramme.

- **Die Lösung, die Anschlusszellen:** Für jede Funktion, die ein Baustein anbietet, gibt es eine Zelle mit **fester, dauerhaft definierter Adresse**, die Anschlusszelle (beim Aufruf durch ein Programm auch Ansprungzelle genannt). Die Adressen stehen in BIS-A.1 (Eingabe- und Hilfsprogramme, 201 bis 430), in BIS-A.2 (Rechen-, Ausgabe- und Satzeingabeprogramme, 7540 bis 7575) und für die Unterprogramme des Minimalprogramms selbst in BIS-A.4. Sie sind auch für Bausteine festgelegt, die noch nicht programmiert sind.
- **Pflicht des Bausteins:** Beim Laden schreibt der Baustein in seine Anschlusszelle einen **unbedingten Sprung auf den Beginn seines Unterprogramms** (`000 <UP-Beginn>`). Mehr braucht er nicht zu tun.
- **Aufruf:** Das Programm springt die Anschlusszelle mit Rückkehrabsicht an (`025 <Anschlusszelle>`) und landet über den eingetragenen Sprung im Unterprogramm, ohne zu wissen, wo der Baustein liegt. Aufrufer kann auch das Minimalprogramm sein (Kennzeichen K_1, siehe BIS-A.1).
- **Baustein nicht geladen:** Wird ein Baustein nicht benötigt oder nicht geladen, ist seine Anschlusszelle **undefiniert**. Das muß der Bediener des Rechners koordinieren: Er lädt alle Bausteine, deren Funktionen das Programm aufruft. Das Programm prüft das nicht.
- **Minimalprogramm:** Es liegt fest und braucht keinen Eintrag. Seine sechs Ansprungzellen sind 0200, 0220 und 0240 (Triaden-, Adreß- und Befehlsdruck; dort steht der Beginn des Unterprogramms selbst) sowie 0262, 0263 und 0316 (Eingabe eines Wortes mit Kennzeichen, Eingabe eines Programmstückes, Adresseneingabe; dort steht je ein unbedingter Sprung, `000 kz_anf` bzw. `000 ae_anf`). Die Zelle 0001 (Sprung zum Druck von BZ und AC) ist kein Unterprogrammaufruf, sondern wird über die Tastenfolge angesprungen.

BIS-2 Beschreibung der Bausteine

BIS-2.1 Das kleine Eingabeprogramm (KEP): Voraussetzung des BIS

- **Lage:** Das KEP befindet sich auf Spur 0 der Trommel und belegt die Adressen 0000_8 … 0037_8 (die 32 Zellen der Spur 0). Es wird über den Speicherplatz 0002 angesprungen.
- **Zweck:** Es ermöglicht die Eingabe leistungsstaerkerer Programme als bitweise notierte und abgelochte Programme. Ohne dieses Programm wäre die Eingabe nur mit technischen Mitteln unter großem Aufwand möglich.
- **Schutz:** Das KEP sollte durch Sperrung der Spur 0 grundsätzlich gegen Überschreiben geschützt sein (Bahnsperrenschalter; bei Hauptschalter EIN ist Spur 0 immer gesperrt). Die Eingabe des KEP selbst ist bei vollständig gelöschtem Speicher nur über eine Hilfstastatur möglich (dafür dient die Serviceeinheit).
- **Systemunabhängig:** Es ist systemunabhängig und kann daher von allen Systemen gleichermaßen verwendet werden.
- **Zeichenvorrat:** Das KEP verarbeitet nur die Ziffern „0" und „1", die Zeichen Punkt „.", Doppelpunkt „:" und Zwischenraum „ZR" (ZR, ZWR und Zwischenraum bezeichnen dasselbe Zeichen, die Leertaste). Alle Wörter müssen demzufolge dual dargestellt sein. Anmerkung: Die Vorlage nennt hier zusätzlich das Komma „,"; das ist nach Angabe des Projektinhabers ein Fehler der Vorlage und gilt nicht. Es gibt **kein Eingabekennzeichen**, und eine **Adressenänderung ist nicht möglich**.
- **Eingabe von Worten:** Alle Wörter müssen entsprechend ihrer internen dualen Darstellung durch Kombinationen von „0" und „1" eingegeben werden; es kann mit dem ersten von Null verschiedenen Bit begonnen werden. Das Wort wird im Akkumulator aufgebaut. Als Schlusszeichen dienen Punkt und Doppelpunkt, der Zwischenraum dient zur Korrektur.
- **Wirkung der Schlusszeichen:** Das Schlusszeichen „:" speichert die im Akkumulator aufgebaute Information als Befehl auf einem festen Speicherplatz (Aufbau des Leitbefehls) und setzt anschließend die Eingabe fort. Das Schlusszeichen „." führt den Leitbefehl aus; ist er kein Sprungbefehl, wird anschließend seine Adresse um eine Einheit erhöht und zur Eingabe eines neuen Zeichens übergegangen. Beispiel: Ist der Leitbefehl ein Transportbefehl $a_1\ a_2\ a_3\ a_4$ `240`, so wird durch den Punkt das zuvor im Akkumulator aufgebaute Wort nach der Zelle $a_1\ a_2\ a_3\ a_4$ abgespeichert; die Erhöhung des Leitbefehls um eine Adresseneinheit gewährleistet die Abspeicherung in natürlicher Reihenfolge, die nur durch die Eingabe eines neuen Leitbefehls unterbrochen werden kann. Soll der im Akkumulator aufgebaute Befehl sofort abgearbeitet werden, müssen als Schlusszeichen „:" und „." gegeben werden.
- **Korrektur:** Mit „ZR" können Tipp- und Lochfehler korrigiert werden, solange noch kein Schlusszeichen gegeben wurde; der Akkumulatorinhalt wird gelöscht und das Wort kann nochmals eingegeben werden.
- **Zugehörigkeit:** Das KEP liegt auf Spur 0 und zählt zur Gesamtbelegung des Minimalprogramms (0000–0337). Es ist aber nicht Teil dessen, was geladen wird, sondern Voraussetzung des BIS: Das separat ladbare Minimalprogramm (0040–0337) wird erst mit Hilfe des KEP geladen.

- **Aufgabe im BIS:** Mit Hilfe des KEP wird das auf dem Lochband dargestellte **Minimalprogramm**, der Grundbaustein des BIS, eingegeben.

Der Leitbefehl des KEP liegt auf Speicherplatz 7723. Die Belegung der Zellen steht im folgenden Beispiel.

Beispiel: Speicherbelegung des KEP (Zellen 0000 … 0037). Inhalt oktal (11 Ziffern: Adreßteil, dann BAZ); „R 300" und „FS" bezeichnen den Eingabecode, für den die Zelle gilt (FS = Fernschreibcode). Zellen, die in dieser Wiedergabe fehlen (0001, 0003 bis 0013), sind in dem Beispiel nicht belegt.

Anmerkung (Festlegung des Projektinhabers): Die Tabelle zeigt den Zustand nach dem Laden des KEP auf die Magnettrommel, also vor dem Laden des Minimalprogramms. Zelle 0000 enthält dann den Sprung zu 0002 (Anfang des KEP). Das Minimalprogramm v6 hat kein Hauptprogramm; es setzt Zelle 0000 auf den Sprung `000 0262` zur Eingabe eines Wortes mit Kennzeichen (Zelle 0001: Sprung zum Druck von BZ und AC), sein Lochstreifen enthält also diesen Sprung nach 0000 (Änderung 24.10.2026). Seine Unterprogramme und Ansprungzellen stehen in BIS-A.4, seine Arbeitszellen in BIS-A.3. Die „Hauptprogramm"-Rolle hat die Eingabe mit Kennzeichen (Ansprung 0262/0263, Anfang `kz_anf` 0040). Die Zellen 0002 und 0022 bis 0024 bleiben dabei erhalten: Sie sind die Initialisierung des KEP (0022 und 0023 setzen die Hauptleitadresse in Zelle 7724 auf `0014 000`, weil sie beim frisch geladenen KEP noch nicht gesetzt ist; 0024 setzt ⟨AC⟩ := 0 und springt zur Eingabe). Überschrieben wird außerdem die im KEP unbelegte Zelle 0001: Dort steht dann ein unbedingter Sprung zum Druck von BZ und AC (Einsprung von HG M↓ E M↑ R, s. BIS-2.2.12). Alle übrigen Zellen des KEP bleiben unverändert.

Zelle	Inhalt	BAZ / Adresse	Bedeutung
0000	00000002000	000 0002	Sprung zum Programmanfang des KEP
0002	00000022000	000 0022	Sprung zur KEP-Initialisierung (Zelle 0022)
0014	00007723211	211 7723	⟨AC⟩ := ⟨7723⟩ (Leitbefehl erhöhen)
0015	00000025210	210 0025	⟨AC⟩ := ⟨AC⟩ + 1 Adresseneinheit
0016	00007723240	240 7723	⟨7723⟩ := ⟨AC⟩ = Leitbefehl
0017	00000030001	001 0030	⟨AC⟩ := 0, Sprung zur Eingabe
0020	00000000000	Konstante	„0" im R 300-Code
0021	40000000000	Konstante	„1" im R 300-Code
0022	00000037211	211 0037	⟨AC⟩ := 0014 000 (Initialisierung der Hauptleitadresse)
0023	00007724240	240 7724	⟨7724⟩ := 0014 000 (Hauptleitadresse setzen)
0024	00000030001	001 0030	„ZWR" im FS-Code, ⟨AC⟩ := 0
0025	00000001000	Konstante	Adresseneinheit 1
0026	00000000000	Konstante	„0" im FS-Code
0027	40000000000	Konstante	„1" im FS-Code (Zeile im Listing verdruckt, ergänzt)
0030	00000020010	010 0020	Eingabe-Halt für 0, 1, „:", „." und ZWR
0031	00000030006	006 0030	⟨AC⟩ := Vzl(⟨AC⟩) (Vzl = zyklische Verschiebung links), Sprung zur Eingabe
0032	00000030001	001 0030	„ZWR" im R 300-Code, ⟨AC⟩ := 0
0033	00007723000	000 7723	„." im R 300-Code, Sprung zur Ausführung des Leitbefehls
0034	00007723000	000 7723	„." im FS-Code
0035	00000016000	000 0016	„:" im R 300-Code, Leitbefehl speichern
0036	00000016000	000 0016	„:" im FS-Code
0037	00000014000	Konstante	0014 000

Die Eingabeschleife (Zelle 0030) spricht die Zelle „Adreßteil ∨ Eingabeadresse" an.

BIS-2.2 Das Minimalprogramm

Aufgabe. Da beim C 8205 nur die Eingabe **eines Zeichens** fest verdrahtet ist, sind zur Eingabe von Worten (Befehle, Zahlen oder Adressen) Leseprogramme erforderlich. Das Minimalprogramm enthält solche Routinen bzw. stellt den Anschluss zu anderen Bausteinen des BIS her. Mit Hilfe des Minimalprogramms werden **Adressen und Programme** eingegeben. Die Darstellung dieser Informationen erfolgt mit wenigen Ausnahmen im **Oktalsystem**.

Abgrenzung der Eingabearten. Bei der Eingabe wird unterschieden zwischen

- der Eingabe von **Adressen, Befehlen und Konstanten**, die mit den einzugebenden Programmen in Verbindung stehen, und
- zu verarbeitenden **Daten** (im Allgemeinen Dezimalzahlen).

Das Minimalprogramm steuert nur die Eingabe von Befehlen, Adressen und Konstanten. Die Eingabe von zu verarbeitenden Daten erfolgt über die Anwenderprogramme durch spezielle Eingabebausteine.

Umfang und Bestandteile. Das Minimalprogramm (MP) bildet den Grundbaustein des BIS und belegt die Speicherplätze 0–337. Anmerkung: Da Zahlenangaben im Projekt oktal gelesen werden, sind das 0000 … 0337, genau die Bahnen 0 bis 6 (7 · 32 = 224 Zellen); Spur 0 gehört dazu. Zum MP gehören neben anderen Programmen das Kleine Eingabeprogramm und ein Programm für die Eingabe mit Kennzeichen. Das Programm für die Eingabe mit Kennzeichen besteht aus einer Reihe von Eingabeunterprogrammen. Auf Spur 0 liegt das Kleine Eingabeprogramm (KEP).

Bezeichnung. Das Minimalprogramm als Ganzes ist der Baustein B 1.1. Die Unterprogramm-Bausteinbezeichnungen B 1.1/5 und B 1.1/8 bezeichnen seine Ausführungen für 5-Kanal-Lochstreifen (Fernschreibcode, FS-Code) und für 8-Kanal-Lochstreifen (R 300-Code). In diesem Dokument wird nur die 8-Kanal-Ausführung B 1.1/8 (R 300-Code) beschrieben; die 5-Kanal-Ausführung ist nicht aufgenommen.

Welche Kennzeichen das MP selbst verarbeitet. Von den als erstes Kennzeichen definierten Zeichen verarbeitet das MP nur K_1 = 0, 1, 2 sowie K_1 = „ZR" und K_1 = „/". Es enthält also nur Leseprogramme für abzuspeichernde und für sofort auszuführende Maschinenworte, für Hilfsadressen sowie für die Zeichen „ZR" und „Schrägstrich (/)". Für alle übrigen zugelassenen Kennzeichen (K_1 = 3 bis 9, +, B, D, „:" und BM) enthält das MP lediglich Anschlussbedingungen für die diesen Kennzeichen entsprechenden Eingabeprogramme. Soll z. B. eine Dezimalzahl mit K_1 = 3 eingegeben werden, muss vorher das Leseprogramm für Dezimalzahlen abgespeichert werden. Wird als erstes Kennzeichen ein nicht als K_1 definiertes Zeichen eingegeben, geht das MP zur Eingabe eines neuen K_1 über.

Die vom MP verarbeiteten Kennzeichen K_1 im Überblick:

- K_1 = 0: Eingabe von Triaden und Befehlen
- K_1 = 1: Eingabe von Sofortbefehlen
- K_1 = 2: Eingabe von Hilfsadressen
- K_1 = „ZR" (Zwischenraum): Löschen
- K_1 = „/" (Schrägstrich): Lochbandschlußzeichen

BIS-2.2.1 Allgemeine Eingaberegeln

- **Eingabecodes und Geräte:** Das BIS enthält Programme für die Eingabe im 5-Kanal-Fernschreibcode (Internationales Telegraphenalphabet Nr. 2) und im R 300-Code (8-Kanal-Code). Die Eingabe kann über die Tastatur, die Schreibmaschine oder die beiden Lochbandleser erfolgen.
- **Eingabekennzeichen:** Die Eingabeorganisation des BIS erlaubt mit Hilfe von Eingabekennzeichen die Eingabe von Worten unterschiedlicher Bedeutung (Befehle, Zahlen in verschiedenartiger Darstellung). Außerdem enthält das BIS Unterprogramme, die Zahlen eines bestimmten Typs ohne Kennzeichen einlesen (Sätze, K_1 = 8 und 9).
- **Wortmarke (WM):** Die einzugebenden Informationen werden im Allgemeinen durch eine Wortmarke abgeschlossen. In der 5-Kanal-Version ist die Wortmarke das Zeichen Punkt „.". In der 8-Kanal-Version wird bei Maschinenbefehlen, Externbefehlen und Adressen der Punkt verwendet, bei Zahlworten kann der Punkt oder der Tabulator (Tab) das Wortende markieren. Anmerkung (Erläuterung des Projektinhabers): Die gedruckte Originaldokumentation kann ein Tab nicht als sichtbares Zeichen darstellen und schreibt deshalb als Wortmarke immer den Punkt. Eine Taste gibt immer nur einen Code ein (am Bedienpult erzeugt die Taste WM den Tab-Code); die Software akzeptiert bei Zahlworten aber beide Codes. In diesem Dokument steht „." für die Wortmarke, bei Zahlworten im R 300-Code gilt Tab gleichwertig.
- **Führende Nullen** können bei Adressen, Befehlen und Zahlen im Allgemeinen weggelassen werden. Eine Ausnahme bilden die Externbefehle (K_1 = +).
- **Vorzeichen:** Bei der Eingabe von Zahlen in allen Darstellungen darf ein positives Vorzeichen nicht eingegeben werden. Ein negatives Vorzeichen kann an beliebiger Stelle, auch innerhalb der Zahl, vor der Wortmarke stehen.
- **Relative Programmierung:** Fast alle Programme werden relativ programmiert. Dadurch kann ein aufgestelltes Programm auf einen beliebigen Speicherbereich eingegeben werden. Man nummeriert ein Programm von der Adresse 0000 aus und markiert die Befehle, deren Adresse sich auf diese Anfangsadresse bezieht. Bei der

Eingabe wird der Adreßteil jedes so markierten Befehls um die Adresse erhöht, ab der die Abspeicherung beginnt. Diese Adresse heißt beim C 8205 **Hauptleitadresse** (= 1. Leitadresse). Sie bestimmt, ab welcher Adresse das Programm gespeichert wird, und dient zur Adressenänderung 1. Art. Außerdem stehen acht weitere **Leitadressen** (2. bis 9.) zur Verfügung, die ausschließlich für die Adressenänderung 1. Art benutzt werden, z. B. als Anfangsadressen von Zahlenfeldern, die das Programm benötigt, und zur Parameterversorgung von Unterprogrammen.

BIS-2.2.1a Bemerkungen zur Eingabe (Ergänzung 11.10.2026)

- **Doppelpunkt und Satzschlusszeichen (nur Satzeingabe):** Bei der Satzeingabe im R 300-Code wirkt „:“ wie die Blockmarke „BM“. Die Satzschlusszeichen „SM/WRZL“, „BM“ und „:“ schließen die Funktion der Wortmarke nicht ein: Vor der Satzmarke ist die Marke des letzten Wortes anzugeben.
- **Löschen:** Ein Wort und ein Satz können stets gelöscht werden, solange nicht die entsprechenden Schlusszeichen eingegeben sind. Eine Zahl, ein Befehl, ein Externbefehl oder eine Adresse wird mit dem Zeichen Zwischenraum „ZR“ gelöscht; ein Satz wird in der 8-Kanal-Version mit „Irrung Satz (IS)“, „Irrung Block (IB)“ oder dem Apostroph gelöscht. Das Irrungszeichen kann an beliebiger Stelle vor dem Schlusszeichen gegeben werden.
- **Neueingabe nach dem Löschen:** Ein gelöschter Satz beginnt immer wieder mit der ersten Zahl (auch bei K_1 = 8 und K_1 = 9). Ein Wort mit Kennzeichen wird dagegen mit dem ersten Kennzeichen neu eingegeben.
- **WR, GB, KB:** Im 8-Kanal-Code hat das Auftreten der Zeichen „WR“, Großbuchstaben „GB“ und Kleinbuchstaben „KB“ zwischen zwei Zahlen keine Wirkung. Tritt hier „ZV“ auf, so ist anschließend mit „ZR“ zu löschen.
- **ZV (Zeilenvorschub) und BM (Blockmarke):** Beide Zeichen stehen seit dem 11.10.2026 in Tabelle 1 (Zeilen 52 und 53, Kodetabellen 1.4). Sie haben dieselbe Eingabeadresse 072 und unterscheiden sich im Lochbild (ZV v u p 8 2, BM w v u 8 2) und in der Sprung-Umstellung (ZV nein, BM ja). BM ist im R 300-Code Schlusszeichen (BIS-2.2.9).

Stand im Minimalprogramm v6 (Quelltext `pj2-bis_minimalprogramm_v6.asm`, Tests `pj2-bis_v6_kz_*`, `pj2-bis_v6_ae_*` und `pj2-bis_v6_ps_*`, Stand 24.10.2026): Es gibt kein Hauptprogramm und keinen Sprung nach 0000 bei unzulässigen Zeichen. ZR verwirft das gerade gelesene Wort bzw. die Adresse und gibt ein neues K_1 frei (das Unterprogramm wird nicht verlassen). Doppelpunkt und BM sind das Schlusszeichen: Sie beenden die Eingabe an jeder Stelle, auch mitten im Wort (das Wort wird dann nicht gespeichert), und kehren zum Aufrufer zurück. „Irrung Zeichen“ überspringt der Leser (Kodetabellen 1.7, Punkt 4); die davor gelesenen Ziffern bleiben. „+“ ist nicht definiert und wirkt wie die Ziffer 0; die Ziffern 8 und 9 zählen als Werte 8 und 9. K_1 = 1 mit sofortigem Punkt ist der Befehl 0 (Sprung nach 0000). „0 .“ und „1 .“ nehmen den Punkt als K_2 = 0, „2 .“ ist wie `20 00000000000` (RA := 0). WR, GB, KB, ZV und IS gibt es auf der Bedienpulttastatur nicht; sie sind im Minimalprogramm v6 nicht vorgesehen. Nicht umgesetzt: Satzeingabe und „IB“ (nicht in Tabelle 1).

BIS-2.2.2 Eingabekennzeichen K_1 und K_2

Zum Unterscheiden zwischen Adressen, Befehlen und Konstanten wurden besondere Eingabekennzeichen geschaffen. Diese Kennzeichen K_1 und K_2 gehen jeder Eingabe über das Minimalprogramm voran. Die externe Notierung eines Befehls sieht deshalb vollständig so aus:

K_1 K_2 a_1 a_2 a_3 a_4 BAZ

- **K_1** (erstes Kennzeichen) ist das Unterscheidungsmerkmal für die verschiedenen möglichen Arten einer Information; entsprechend K_1 wird das eingelesene Wort interpretiert und weiterverarbeitet. Als 1. Kennzeichen sind definiert:
  - die Ziffern 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
  - die Tastatur-Tasten B und D (nicht die SW-Tasten B und D !),
  - die Zeichen Plus (+), Zwischenraum (ZR), Schrägstrich (/), Doppelpunkt (:) und Blockmarke (BM); BM tritt nur im R 300-Lochbandcode auf.
- **K_2** (zweites Kennzeichen) kann die Werte 0 bis 9 annehmen. Bei Befehlen dient es der Adressenmodifikation 1. Art; bei K_1 = 2 wählt es die Art der Hilfsadresse.
- Alle definierten Kennzeichen K_1 und K_2 haben bei der Eingabe im Fernschreibcode und im R 300-Code die gleiche Bedeutung.
- Eingabekennzeichen werden **nicht abgespeichert**. Sie steuern lediglich die Eingabe.

Beschrieben sind K_1 = 0 bis 9 und die Zeichen +, B, D, ZR und /. Das Minimalprogramm selbst verarbeitet nur K_1 = 0, 1, 2, ZR und /; die Leseprogramme für die übrigen Kennzeichen sind eigene Eingabebausteine und müssen vorher abgespeichert sein.

Übersicht der Kennzeichen K_1 = 0 bis 9:

K_1	Bedeutung	Interne Form	Externe Darstellung
0	Maschinenwort zur fortlaufenden Abspeicherung	Oktalwort, bis 11 Oktalziffern	`0` K_2 t_{10} … t_0 `.` (bis 11 Oktalziffern); freie Zelle: `0.` (Kurzform für `0 0 00000000000 .`, also K_2 = 0 und elf Nullen)
1	Maschinenbefehl zur sofortigen Abarbeitung	Befehl	`1` K_2 a_1 a_2 a_3 a_4 `BAZ .` (= `1` K_2 t_6 … t_0 `.`)
2	Adresseneingabe (K_2 = 0 Relativadresse, K_2 = 1 Hauptleitadresse, K_2 = 2 … 9 2. … 9. Leitadresse)	Adresse	`2` K_2 b_1 b_2 b_3 b_4 a_1 a_2 a_3 a_4 `.` (allgemeine Form); mit b_1 … b_4 = 0: `2` K_2 a_1 a_2 a_3 a_4 `.` (Kurzform)
3	Konstante als Dezimalzahl	Gleitkommazahl	`3 (-) a, b .` bzw. `3 (-) a .`
4	Konstante als Gleitkommazahl	Gleitkommazahl	`4 (-)` d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8 `. (-) e .`
5	Konstante als Dezimalzahl	Festkommazahl, Komma hinten (ganze Zahl)	`5 (-)` d_9 d_8 d_7 d_6 d_5 d_4 d_3 d_2 d_1 d_0 `.` (zehn Ziffern)
6	Konstante als Dezimalzahl, nur ganze positive	Tetradendarstellung	`6` d_7 d_6 d_5 d_4 d_3 d_2 d_1 d_0 `.`
7	wie 5, höhere Lesegeschwindigkeit	wie 5	`7 (-)` d_7 d_6 d_5 d_4 d_3 d_2 d_1 d_0 `.` (acht Ziffern; bei Zahlworten im R 300-Code Punkt oder Tab)
8	Dezimalzahlen im Satz (ohne Kennzeichen je Zahl)	Gleitkommazahlen	Kennzeichen nur vor dem ersten Wort (Zahlenformat wie bei K_1 = 3)
9	Dezimalzahlen im Satz (ohne Kennzeichen je Zahl)	Festkommazahlen, Komma hinten	Kennzeichen nur vor dem ersten Wort (Zahlenformat wie bei K_1 = 5)

Übersicht der Kennzeichen K_2:

K_2	Bedeutung
0	Relativadresse bei der Adresseneingabe; bei Maschinenworten und -befehlen keine Adressenmodifikation (die angegebene Adresse ist absolut)
1	1. Leitadresse = Hauptleitadresse; bei Maschinenworten und -befehlen Adressenmodifikation mit der 1. Leitadresse
2 … 9	2. … 9. Leitadresse; bei Maschinenworten und -befehlen Adressenmodifikation mit der 2. … 9. Leitadresse

K_2 = 0 bis 9 gelten sowohl bei K_1 = 0 und 1 (Adressenmodifikation des eingelesenen Wortes) als auch bei K_1 = 2 (Eingabe der Hilfsadresse selbst).

BIS-2.2.3 K_1 = 0: Eingabe eines Maschinenwortes zur fortlaufenden Abspeicherung

Nach der Eingabe des ersten und zweiten Kennzeichens können maximal **11 Oktalziffern** t_{10} … t_0 (z. B. ein Befehl) eingegeben werden: `0` K_2 t_{10} t_9 t_8 t_7 t_6 t_5 t_4 t_3 t_2 t_1 t_0 `.` Nach der letzten Oktalziffer muss das Schlusszeichen Punkt „." folgen. Die Eingabe kann mit der ersten von Null verschiedenen Ziffer beginnen. Soll in der Reihenfolge der fortlaufenden Abspeicherung eine Speicherzelle **frei gehalten** (auf Null definiert) werden, so genügt die Darstellung `0.` (Null – Punkt): Nach der Eingabe des ersten Kennzeichens K_1 = 0 kann sofort die Wortmarke gegeben werden.

Anmerkung (Stand im Quelltext, 11.10.2026): „0.“ ist umgesetzt. Es bedeutet 0000 0000 000 (Angabe des Projektinhabers: Kommt nur eine Null, sind die folgenden zehn Oktalziffern ebenfalls null). Nach dem Merken von K_1 setzt `kz_h2` die Tabellenzelle des Punktes auf `000 kz_p0`; kommt der Punkt als K_2-Zeichen, setzt `kz_p0` das Wort und K_2 auf 0 zurück und springt zum Wortende `kz_wend`. K_1 = 1 oder 2 mit sofortiger Wortmarke bleibt unzulässig (Sprung nach 0000).

Die eingelesenen Oktalziffern werden ohne Konvertierung abgespeichert und, falls sie einen Internbefehl darstellen, erst später abgearbeitet. Ob das Wort als Befehl oder als Zahl aufzufassen ist, hängt von seiner Verwendung ab; dieselbe Oktalzahl kann in einem Programm als Befehl und als Konstante benutzt werden.

In Abhängigkeit vom zweiten Kennzeichen K_2 kann vor der Abspeicherung eine **Adressenänderung** durchgeführt werden. Sie bezieht sich auf die Oktalstellen t_3 bis t_{10}, die damit den Adreßteil eines Maschinenwortes darstellen:

- **$K_2 = 0$:** Das Wort wird ohne Änderung der Oktalstellen t_3 bis t_{10} abgespeichert. Stellt es einen Befehl dar, ist die angegebene Adresse absolut.
- **$K_2 = 1$:** Zu den Oktalstellen t_3 bis t_{10} wird vor der Abspeicherung die Hauptleitadresse (= 1. Leitadresse) addiert.
- **$K_2 = 2 \ldots 9$:** Zu den Oktalstellen t_3 bis t_{10} wird vor der Abspeicherung die 2. … 9. Leitadresse addiert.

BIS-2.2.4 $K_1 = 1$: Maschinenbefehl zur sofortigen Abarbeitung

Nach Eingabe des ersten und zweiten Kennzeichens könnten bis zu 11 Oktalziffern eingegeben werden. Da das eingelesene Wort jedoch als Befehl interpretiert wird, ist nur die Eingabe von **maximal 7 Ziffern** sinnvoll: $1\ K_2\ t_6\ t_5\ t_4\ t_3\ t_2\ t_1\ t_0\ .$ (= $1\ K_2\ a_1\ a_2\ a_3\ a_4$ BAZ .). Wie bei $K_1 = 0$ kann mit der ersten Ziffer ungleich Null begonnen werden, und nach der letzten Ziffer muss die Wortmarke Punkt gegeben werden. Das zweite Kennzeichen K_2 hat dieselbe Bedeutung wie bei $K_1 = 0$.

Der Befehl (z. B. Sprungbefehl zum Start von Programmen) wird nach seiner Eingabe in einer **festen Zelle des Systems** gespeichert und anschließend **sofort ausgeführt**. Ist der Sofortbefehl kein Sprungbefehl, geht der Rechner nach der Ausführung in den Zustand „US" (unbedingter Stopp). Bei Betätigung der Starttaste wird die Adresse des Sofortbefehls um eins erhöht, der dadurch entstandene Befehl sofort ausgeführt und der Rechner geht wieder in den Zustand „US". Mit der Tastenkombination „HGR" kann zur Eingabe eines neuen ersten Kennzeichens zurückgekehrt werden. So kann z. B. durch einen Sofort-Lesebefehl der Inhalt einer Speicherzelle in den Akkumulator gebracht und über ein Hilfsprogramm ausgegeben werden.

Anmerkung: Die „feste Zelle des Systems" ist beim KEP die Zelle 7723 (Leitbefehl). Ob das Minimalprogramm dieselbe Zelle benutzt, sagen die vorliegenden Seiten nicht.

BIS-2.2.5 $K_1 = 2$: Adresseneingabe (Hilfsadressen)

Die Adresseneingabe ist notwendig, um z. B. die absolute Anfangsadresse eines Programms festzulegen, die für die Abspeicherung von Informationen verantwortlich ist. Mit Hilfe des zweiten Kennzeichens K_2 unterscheidet das Minimalprogramm, um welche Art von Adresse es sich handelt ($K_2 = 0$ bis 9 sind zugelassen).

Eingegeben werden nach den beiden Kennzeichen maximal **8 Oktalziffern**, die zwei oktale Adressen darstellen: $2\ K_2\ b_1\ b_2\ b_3\ b_4\ a_1\ a_2\ a_3\ a_4$. Führende Nullen brauchen nicht eingegeben zu werden, als Wortmarke dient der Punkt. Die 8 Oktalziffern werden auf festen Speicherplätzen in den Oktalstellen t_3 bis t_{10} gespeichert. Die Möglichkeit, Hilfsadressen mit 8 Ziffern einzugeben, bietet den Vorteil, dass auch Änderungen mit den ersten vier Stellen $t_{10}\ t_9\ t_8\ t_7$ einer Oktalzahl durchgeführt werden können (vgl. $K_1 = 0$).

Anmerkung (Erläuterung des Projektinhabers): Die allgemeine Form $2\ K_2\ b_1\ b_2\ b_3\ b_4\ a_1\ a_2\ a_3\ a_4$. kennt $b_1 \ldots b_4$ als obere vier Stellen. Häufig ist $b_1 \ldots b_4 = 0$; dann entfallen sie als führende Nullen und die Eingabe lautet $2\ K_2\ a_1\ a_2\ a_3\ a_4$. (Kurzform, so in der Übersichtstabelle der Quelle). Beide Darstellungen sind gleichwertig. Die Addition einer Leitadresse zu den Oktalstellen $t_3 \ldots t_{10}$ erfolgt zum ganzen Maschinenwort, auch wenn $b_1 \ldots b_4 \neq 0$ ist.

Die Bedeutungen von K_2:

- **$K_2 = 0$, Relativadresse:** Sinnvoll ist nur die Angabe der vier Oktalziffern $a_1\ a_2\ a_3\ a_4$ ($2\ 0\ a_1\ a_2\ a_3\ a_4$.). Mit der Relativadresse kann die Abspeicherung in natürlicher Reihenfolge unterbrochen werden. Sie bewirkt einen Neuaufbau des Abspeicherbefehls (vgl. $K_2 = 1$): Die Abspeicherung wird ab der Adresse fortgesetzt, die sich aus der Summe von Hauptleit- und Relativadresse ergibt. Die Hauptleitadresse wird dabei nicht verändert und kann weiterhin verwendet werden. Die Relativadresse steht danach nicht mehr zur Verfügung.
- **$K_2 = 1$, Hauptleitadresse (1. Leitadresse):** Sie kann mit maximal 8 Ziffern eingegeben werden. Mit ihr wird der Abspeicherbefehl für die einzugebenden Worte aufgebaut. Die Speicherung beginnt bei der durch $a_1\ a_2\ a_3\ a_4$ angegebenen Zelle; $b_1\ b_2\ b_3\ b_4$ haben keinen Einfluss auf die Abspeicherung eines Wortes. Nach jeder erfolgten Eingabe eines zu speichernden Wortes wird die Adresse des Abspeicherbefehls um eins erhöht, sodass eine Abspeicherung in natürlicher Reihenfolge gewährleistet ist. Sie wird durch die Eingabe einer Relativadresse oder einer neuen Hauptleitadresse unterbrochen. Die Hauptleitadresse dient außerdem zur Adressenänderung
  1. Art: Bei einer 8-stelligen Hauptleitadresse sind bei der Eingabe von Maschinenworten Adressenänderungen nicht nur mit der eigentlichen Adresse eines Befehlswortes ($t_6\ t_5\ t_4\ t_3$) möglich, sondern auch mit den Oktalstellen $t_{10}\ t_9\ t_8\ t_7$ (vgl. $K_1 = 0$). Hauptleitadresse und Abspeicherbefehl werden in festen Zellen gespeichert; die Hauptleitadresse wird im Allgemeinen erst durch die Eingabe einer neuen Hauptleitadresse zerstört.
- **$K_2 = 2 \ldots 9$, 2. … 9. Leitadresse:** Sie können wie die Hauptleitadresse maximal 8 Oktalziffern umfassen ($2\ K_2\ b_1\ b_2\ b_3\ b_4\ a_1\ a_2\ a_3\ a_4$.). Sie haben keinen Einfluss auf die laufende Speicherung von Worten und werden ausschließlich zur Adressenänderung verwendet.

BIS-2.2.6 $K_1 = 3$ bis 9: Konstanten und Datensätze

Die Kennzeichen $K_1 = 3 \ldots 9$ dienen der Eingabesteuerung von Konstanten verschiedener Darstellungsarten. Hierbei folgt dem Kennzeichen K_1 **kein zweites Kennzeichen**, sondern unmittelbar die jeweilige Konstante (keine Adreßrechnung). Diese Kennzeichen verarbeitet nicht das Minimalprogramm selbst, sondern die zugehörigen Eingabebausteine.

- **$K_1 = 3$:** Eingabe einer Konstanten in Form einer Dezimalzahl, die intern in eine Zahl in Gleitkommadarstellung umgewandelt wird. Externe Darstellung: `3 (-) a, b .` bzw. `3 (-) a .`
  - Eingabezahlenbereich: $|z| \leq (1 - 10^{-8}) \cdot 10^{8}$. Nach der ersten von Null verschiedenen Ziffer dürfen höchstens noch 7 Ziffern stehen. Bei Zahlen mit $|z| < 1$ können zwischen Komma und erster von Null verschiedener Ziffer bis zu acht Nullen auftreten.
  - Die Zahl Null wird mit `3 0 .` eingegeben.
- **$K_1 = 4$:** Eingabe einer Zahl in Gleitkommadarstellung. Eine Gleitkommazahl umfasst maximal **14 Zeichen** (erstes Kennzeichen, negatives Vorzeichen der Mantisse, maximal acht Mantissenziffern, Mantissenschlusszeichen, negatives Vorzeichen des Exponenten, Betrag des Exponenten E, Exponentenschlusszeichen). Externe Darstellung: `4 (-) d1 d2 d3 d4 d5 d6 d7 d8 . (-) e .`
  - Ein negatives Vorzeichen des Exponenten muss vor diesem eingegeben werden.
  - Alle Gleitkommazahlen mit Ausnahme der Null sind normalisiert einzugeben, d. h. in jedem Fall muss $d_1 \neq 0$ sein. Das Mantissenschlusszeichen kann nach der letzten Ziffer ungleich Null gegeben werden.
  - Die Null ist als `4 0 .` einzugeben. Der Eingabezahlenbereich ist wie bei $K_1 = 3$ $|z| \leq (1 - 10^{-8}) \cdot 10^{8}$.
- **$K_1 = 5$:** Eingabe einer Dezimalzahl, intern Konvertierung in eine **Festkommazahl, Komma hinten**. Es können ganze Zahlen mit

  d ≤ $2^{31} - 1$ = 2 147 483 647 eingegeben werden. Die Eingabe umfasst maximal 13 Zeichen (erstes Kennzeichen, negatives

  Vorzeichen, maximal 10 Dezimalziffern, Wortmarke): `5 (-) d9 d8 d7 d6 d5 d4 d3 d2 d1 d0 .` Anmerkung (Festlegung des Projektinhabers): Die Grenze $2^{31} - 1$ bezieht sich nicht auf das Maschinenwort, sondern auf die softwareseitig realisierte Festkommaarithmetik des BIS. Anmerkung (aus der Quelle): Die Kommastellung der einzugebenden Zahl ist vom Programmierer bzw. bei der Erstellung der Daten zu berücksichtigen.
- **$K_1 = 6$:** Eingabe einer Dezimalzahl, intern Konvertierung in eine Zahl in **Tetradendarstellung**. Es können nur **ganze positive Zahlen** $d \leq 10^{8} - 1$ eingegeben werden. Das Eingabebild umfasst maximal **10 Zeichen** (erstes Kennzeichen, maximal 8 Dezimalziffern, Wortmarke): `6 d7 d6 d5 d4 d3 d2 d1 d0 .` Jede Dezimalziffer wird durch eine Tetrade dargestellt und in dieser Form abgespeichert.
- **$K_1 = 7$:** Eingabe einer Dezimalzahl, intern Konvertierung in eine Festkommazahl, Komma hinten. Eingabezahlenbereich $|d| \leq 10^{8} - 1$. Die Eingabe umfasst maximal **11 Zeichen** (erstes Kennzeichen, negatives Vorzeichen, maximal 8 Dezimalziffern, Wortmarke): `7 (-) d7 d6 d5 d4 d3 d2 d1 d0 .` Nach dem Einlesen und der Darstellung jeder Dezimalziffer durch eine Tetrade wird die Zahl in eine Festkommazahl, Komma hinten, konvertiert. Mit $K_1 = 7$ ist eine schnellere Eingabe von Festkommazahlen, Komma hinten, möglich als mit $K_1 = 5$.
- **Gemeinsame Regel für $K_1 = 5, 6, 7$:** Ein eventuell vorhandenes Komma wird überlesen. Die Zahl wird in jedem Falle als **ganze Zahl** interpretiert und in der entsprechenden Form abgespeichert.
- **$K_1 = 8$:** Eingabe von Dezimalzahlen im Satz (ohne Kennzeichen je Zahl; intern: Konvertierung in Gleitkommazahlen). Nach $K_1 = 8$ können hintereinander Dezimalzahlen ohne erstes Kennzeichen $K_1 = 3$ eingelesen werden (vgl. $K_1 = 3$).
- **$K_1 = 9$:** Eingabe von Dezimalzahlen im Satz (ohne Kennzeichen je Zahl; intern: Konvertierung in Festkommazahlen, Komma hinten). Nach $K_1 = 9$ können hintereinander Dezimalzahlen ohne erstes Kennzeichen $K_1 = 5$ eingelesen werden (vgl. $K_1 = 5$).

Regeln für die Eingabe im Satz ($K_1 = 8$ und 9):

- Die einzugebenden Zahlen müssen **gleichen Typ** sein.
- Sie sind durch Punkt „." (= Wortmarke) voneinander getrennt.
- Die Abspeicherung erfolgt entsprechend der **Hauptleitadresse**, die dem Rechner vor dem ersten Kennzeichen $K_1 = 8$ oder 9 eingegeben werden muss. Sie endet mit einer Satzmarke.
- Der Satz wird im Fernschreibcode durch den Doppelpunkt „:" abgeschlossen. Im R 300-Code kann die Satzeingabe durch die Zeichen Satzmarke (SM, = WRZL), Blockmarke (BM) oder Doppelpunkt (:) beendet werden.

- Nach der Eingabe eines dieser Zeichen ist der Rechner bereit, ein neues erstes Kennzeichen K_1 einzulesen.

BIS-2.2.7 K_1 = +, B, D, ZR, /

Das Minimalprogramm verarbeitet von diesen Kennzeichen nur ZR und / selbst; für +, B und D enthält es nur Anschlussbedingungen.

- **K_1 = + (Externbefehl):** Externbefehle sind Pseudobefehle. Sie umfassen maximal **7 Zeichen**: erstes Kennzeichen +, zwei oktale Ziffern O_1 O_2, maximal 3 Dezimalziffern p_1 p_2 p_3 und die Wortmarke Punkt: $+\ 0_1\ 0_2\ p_3\ p_2\ p_1$. Die beiden Oktalziffern müssen in jedem Fall angegeben werden, auch bei O_1 = 0. Bei den Parametern p_1, p_2, p_3 können führende Nullen weggelassen werden. Wird z. B. nur p_1 gefordert, lautet das Eingabebild $+\ 0_1\ 0_2\ p_1$. Bei der Eingabe wird der zugehörige Internbefehl aus einer Tabelle ausgesucht und nach Addition der Parameter p_1, p_2 und p_3 abgespeichert. Die Externbefehle behandelt die Quelle in einem besonderen Abschnitt (nicht erfasst).
- **K_1 = B, K_1 = D (Hilfsprogramme):** Mit diesen Kennzeichen werden Bedienungs- und Kontrollprogramme angesprungen. Ein zweites Kennzeichen unterscheidet zwischen den einzelnen Programmen. Die Beschreibung der Programme steht in einem eigenen Abschnitt der Quelle (nicht erfasst). Anmerkung: Die Tasten B und D am Bedienpult sind Sondertasten für den Programmbaustein-Ansprung des BIS.
- **K_1 = ZR (Löschen):** Das erste Kennzeichen ZR dient zur Gliederung des Lochbandes. Bei Eingabe von ZR wird der Inhalt des Akkumulators gelöscht und zu einer neuen Eingabe übergegangen. Die Verwendung von ZR beschreibt die Quelle in einem eigenen Abschnitt (nicht erfasst).
- **K_1 = / (Eingabe-Ende):** Dieses Zeichen steht im Allgemeinen am Ende eines Lochbandes. Bei eingerastetem Schalter „BS" geht der Automat in bedingten Stopp. Durch Lösen von „BS" kann die Eingabe fortgesetzt werden.

BIS-2.2.8 Eingabeunterprogramme

Bei der Eingabe mit Unterprogrammen wird von einem Hauptprogramm durch den Befehl „Sprung mit Rückkehrabsicht" zu einem Leseprogramm gesprungen, das als Unterprogramm aufgebaut ist. Nach dem Einlesen der Information und eines die Eingabe beendenden Schlußzeichens wird die Abarbeitung des Hauptprogrammes fortgesetzt. Die Eingabe kann über die Tastatur, das Schreibwerk oder die beiden Lochbandleser erfolgen.

BIS-2.2.9 Eingabeunterprogramme des Minimalprogramms

Im Minimalprogramm sind drei Eingabeprogramme enthalten, die als Unterprogramme verwendet werden können:

Eingabeprogramm	Ansprungzelle
Adresseneingabe	316
Eingabe eines Wortes mit Kennzeichen	262
Eingabe eines Programmstückes	263

- **Adresseneingabe:** Nach dem Ansprung des Unterprogramms „Adresseneingabe" ist der Automat bereit, eine maximal achtstellige oktale Adreßkonstante einzulesen. Dabei kann mit der ersten Ziffer ungleich Null begonnen werden. Als Schlußzeichen muß das Zeichen Punkt „." eingegeben werden. Bei einer Adreßkonstante Null ist nur die Eingabe des Schlußzeichens notwendig. Die Konstante steht nach dem Einlesen im Akkumulator und belegt die Wortstellen z_0 bis z_{23}. Bei Verwendung dieses Eingabeunterprogrammes darf das Zeichen „Zwischenraum" nicht gegeben werden, da „ZR" einen Sprung zum Programm „Kennzeicheneingabe" bewirkt. In diesem Fall gibt es bei Angabe einer falschen Adresse keine Korrekturmöglichkeit.
- **Eingabe eines Wortes mit Kennzeichen:** Bei diesem Unterprogramm erfolgt die Interpretation der einzugebenden Informationen entsprechend den Eingabekennzeichen K_1 und K_2. Es wird nur ein Wort in den Akkumulator eingelesen. Die Abspeicherung oder Weiterverarbeitung muß durch das Hauptprogramm erfolgen. Bei Eingabe einer Relativ- oder Hauptleitadresse mit 20 bzw. 21 wird nicht zum Hauptprogramm zurückgekehrt. Der Automat wartet dann auf die Eingabe eines neuen ersten Kennzeichens. Wird eine Leitadresse (2. bis 9.) eingegeben, so liest der Rechner anschließend ein weiteres Wort mit Kennzeichen ein, ehe zum Hauptprogramm zurückgekehrt wird.
- **Eingabe eines Programmstückes:** Bei Verwendung dieses Unterprogramms wird die Regie dem im Minimalprogramm enthaltenen Programm für die Kennzeicheneingabe übergeben. Es können ganze Programme oder auch Datenblöcke eingegeben werden. Die Abspeicherung des Programmstückes erfolgt ab der durch die Hauptleitadresse angegebenen Speicherzelle. Die Hauptleitadresse kann über die Tastatur eingegeben werden, oder sie kann auf dem Lochband stehen.

Die Rückkehr ins Hauptprogramm ist durch Eingabe des Schlußzeichens möglich. Als Schlußzeichen werden im R 300-Code die Zeichen Blockmarke „BM" und Doppelpunkt „:" verwendet.

Anmerkung (Stand im Quelltext v6, 24.10.2026): Die Eingabe eines Programmstückes (0263, `eingabeprogst`) und die Eingabe eines Wortes mit Kennzeichen (0262, `eingabewortkz`) springen beide auf `kz_anf` (0040); es gibt keinen eigenen Code `ps_anf`. Die Unterprogramme enthalten keine Kanalanwahl: Kanal und Hauptleitadresse stellt der Aufrufer bzw. das Band ein (Bedienung `H G K1 R`, Hauptleitadresse `2 1` a_1 a_2 a_3 a_4 `.`, Daten vom Lochstreifen). Aufrufe, Parameterstellen und Prüfskripte aller Unterprogramme: BIS-A.4.

- **Leitbefehl:** `LB` (7600) wird bei jedem Einstieg und nach jeder Änderung neu gebaut: `LB` := (Hauptleitadresse 7611) + Relativadresse `RA` (7742) + 0240.
- **$K_1 = 0$:** Das Wort (mit Adressenänderung nach K_2) wird über `LB` abgespeichert; danach wird `RA` um 01000 (eine Adresseinheit) erhöht und `LB` neu gebaut. „0." ($K_1 = 0$ und sofort die Wortmarke) speichert das Wort 0 (s. BIS-2.2.3, Anmerkung). Die Hauptleitadresse ist Speicherort und Basis der Adressänderung $K_2 = 1$.
- **$K_1 = 2$:** $K_2 = 1$ setzt die Hauptleitadresse und `RA` := 0; $K_2 = 0$ setzt `RA` := Adresse (Speicherung ab Hauptleitadresse + `RA`); $K_2 = 2$ bis 9 setzen die Leitadressen 2 bis 9, die die Abspeicherstelle nicht beeinflussen. Das gilt auch außerhalb der Programmstück-Eingabe.
- **$K_1 = 1$ (Sofortbefehl):** Der Befehl wird auf `XW` (7602) ausgeführt, dahinter steht auf `XW1` (7603) der Stopp. Die Starttaste (R) führt über `XW2` (7604) zur Anwahl von K0 und zum neuen Kennzeichen. Ein Sprungbefehl verlässt das Unterprogramm (so kehrt man mit einem Sofortbefehl an eine andere Stelle des Hauptprogramms zurück).
- **Abschluss einer Eingabe:** Nach der letzten Ziffer (elf bei einem Wort, acht bei einer Adresse) muss der Punkt folgen; weitere Ziffern werden ignoriert (`pt_wait`).
- **Schlusszeichen:** Der Doppelpunkt „:" und das Zeichen BM (Blockmarke, Eingabeadresse 072, Tabelle 1 Zeile 53) beenden das Unterprogramm (Rückkehr zum Aufrufer über `kz_rs`).
- **Zeichen ohne Wirkung:** ZR löscht das Wort und führt zu einem neuen K_1 (`kz_neu`); „/" ist der bedingte Stopp (Schalter BS). „+" ist nicht definiert und wirkt wie die Ziffer 0. WR, GB, KB, ZV und IS gibt es auf der Bedienpulttastatur nicht und sind im Minimalprogramm nicht enthalten.
- **„2 ." (Festlegung des Projektinhabers, 24.10.2026):** Folgt auf $K_1 = 2$ sofort der Punkt statt K_2, so gilt die Eingabe wie `2 0 00000000000`: Adresse 0 mit $K_2 = 0$, also Relativadresse RA := 0 (Speicherung weiter ab Hauptleitadresse + 0). `0 .` und `1 .` nehmen den Punkt ebenfalls als $K_2 = 0$ mit dem Wort 0. Umsetzung: `kz_a2` springt zuerst zu `k2_vor` (0174 bis 0177), das `ae_rs` auf `c_a2n` (0260, Sprung nach `kz_ra0`) stellt und K_2 liest; der Punkt führt über `pt_end` und `ae_end` dorthin, `kz_ra0` setzt RA := 0 und baut den Leitbefehl neu. Test: `pj2-bis_v6_kz_2_punkt`.
- **Nicht umgesetzt:** Wörter mit $K_1 \geq 3$, +, B, D und BM als Kennzeichen (Anschlussbedingungen) bleiben ohne Wirkung; das Programm erwartet ein neues K_1.

BIS-2.2.10 Ausgabe

Da beim C 8205 nur die Ausgabe **eines einzigen Zeichens** fest verdrahtet ist, wurden für die Ausgabe von ganzen Worten Standardprogramme geschaffen. Dabei kann die auszugebende Information verschieden interpretiert werden.

Ansprungzellen. Alle Ausgabeunterprogramme sind über feste Ansprungzellen erreichbar. Außer für die im Minimalprogramm enthaltenen Ausgabeprogramme befinden sich diese Zellen auf der Bahn 7540. Es werden die Zellen 7554 bis 7563 verwendet.

Versorgungszelle. Als Versorgungszelle wird bei allen Ausgabeunterprogrammen das Register R_1 benutzt. Der Aufruf des Unterprogramms braucht nicht direkt nach der Speicherung des auszugebenden Wortes nach R_1 zu erfolgen, wenn man beachtet, daß der Inhalt dieses Registers inzwischen nicht zerstört wird.

Register nach der Ausgabe. Der Inhalt des Akkumulators beim Rücksprung ist belanglos. Durch die Abarbeitung eines Ausgabeprogrammes wird der Inhalt von R_1 nicht verändert. In den Ausgabeunterprogrammen sind keine Anwahlbefehle berücksichtigt.

BIS-2.2.10.1 Ausgabeprogramme des Minimalprogramms

Von den Ausgabeprogrammen wurden als Unterprogramme in das Minimalprogramm aufgenommen (in der Quelle als häufig benutzte Programme zum Druck von Adressen, Befehlen und Triaden bezeichnet):

Ausgabeprogramm	Ansprungzelle
Triadendruck	276
Befehlsdruck	053
Adressendruck	100
Druck Befehlszähler BZ und Akkumulator AC	0001 (Zelle im KEP-Bereich mit Sprung ins Minimalprogramm; die Routine selbst hat keinen festen Platz)

Befehlsdruck (Wortlaut sinngemäß nach der Programmieranleitung, Seitenabbild): Es werden bei diesem Druck die Wortstellen z_0 bis z_{11} nicht berücksichtigt, da diese für die interne Befehlsinterpretation keine Bedeutung haben. Die Befehlsadresse wird mit 4, der Operationsteil mit 3 Oktalziffern gedruckt. Zwischen dem Druck der Adresse und des Operationsteils wird ein Leerschritt ausgeführt. Beispiel: `0172 211`, `7645 442`.

Anmerkung (Festlegung des Projektinhabers, nicht Wortlaut der Quelle): Der zu druckende Befehl wird wie bei allen Ausgabeunterprogrammen in R_1 übergeben (nicht im AC; der Sprung mit Rückkehrabsicht belegt den AC mit dem Rückkehrbefehl). R_1 bleibt unverändert.

BIS-2.2.11 Programmeingabe über das Minimalprogramm

Ansprung des Eingabeprogramms. Das im Minimalprogramm enthaltene Eingabeprogramm für die Eingabe mit Kennzeichen kann auf der Zelle 0000 mit Hilfe der Eingriffstasten angesprungen werden:

- (a) Schalter „H–R“ auf H drücken,
- (b) Taste G drücken,
- (c) Schalter „H–R“ auf R drücken.

Diese Folge heißt „HGR“. Nach dem Drücken von HGR wird über Zelle 0000 das Minimalprogramm angesprungen und der Rechner wartet auf die Eingabe eines ersten Kennzeichens K_1.

Beispiel der Quelle. Die Eingabe eines Programms wird an einem Beispiel dargestellt. In einem Hauptprogramm (HP) werden zwei Unterprogramme verwendet, die mit 1. UP und 2. UP bezeichnet sind. Das 1. UP benutzt seinerseits das 2. UP als Unterprogramm (UUP). Das HP soll ab der Zelle 3000, das 1. UP ab der Zelle 3400 und das 2. UP ab der Zelle 3500 gespeichert werden. Das Hauptprogramm ist relativ auf dem Speicherplatz 0004 anzuspringen.

Nach dem Ansprung des Eingabeprogramms durch HGR sind die Hilfsadressen, falls sie nicht auf dem Lochband stehen, folgendermaßen einzugeben (Schalter „T–S“ auf „T“). Dabei bedeutet HLA die Hauptleitadresse und LA eine Leitadresse (2. bis 9.).

1. **Eingabe des 2. UP:**
   - Eingabe der HLA: `21 3500 .`
   - Einlesen des Bandes: `H K1 R` bei Leser 1, `H K2 R` bei Leser 2.
2. **Eingabe des 1. UP:**
   - Anwahl der Eingabetasten
   - Eingabe der HLA: `21 3400 .`
   - Eingabe der 2. LA: `22 3500 .`
   - Einlesen des Bandes
3. **Eingabe des Hauptprogramms HP:**
   - Anwahl der Eingabetasten
   - Eingabe der HLA: `21 3000 .`
   - Eingabe der 2. LA: `22 3400 .`
   - Eingabe der 3. LA: `23 3500 .`
   - Einlesen des Bandes
4. **Ansprung des Hauptprogramms:**
   - Falls die HLA 3000 nicht zerstört ist, kann das Programm durch den Sofortbefehl `11 4 000.` angesprungen werden.

- Ist die HLA 3000 nicht mehr erhalten, muß ein Sprungbefehl ohne Adressenänderung gegeben werden: `10 3004 000.`

Anmerkung (Lesehilfe, nicht Wortlaut der Quelle): Die Leitadressen entsprechen den Eingaben `2 K₂ …` ($K_2 = 1$ Hauptleitadresse, $K_2 = 2$ … 9 die 2. … 9. Leitadresse), die Sofortbefehle den Eingaben mit $K_1 = 1$. Beim Sofortbefehl `11 4 000.` bewirkt $K_2 = 1$ die Adressenmodifikation mit der Hauptleitadresse, also 3000 + 4 = 3004. Beim Sofortbefehl `10 3004 000.` ist $K_2 = 0$, die Adresse 3004 wird ohne Modifikation benutzt.

Anmerkung (Lesehilfe, nicht Wortlaut der Quelle): K1 und K2 sind die Kanaltasten des Bedienpults (K1 wählt Kanal 1 = Leser 1, K2 wählt Kanal 2 = Leser 2).

BIS-2.2.11a Laden eines Programms: Objektdatei über das Minimalprogramm oder bitweise über das KEP (Ergänzung 21.10.2026)

Anmerkung (Festlegung des Projektinhabers): Ein Programm kann auf zwei verschiedenen Wegen auf die Trommel gelangen. Welcher Weg möglich ist, hängt davon ab, was schon auf der Trommel steht und in welcher Form das Programm vorliegt.

Weg 1: Objektdatei über das Minimalprogramm. Eine echte Objektdatei, wie sie der Assembler als `.obj`-Datei erzeugt, kann das Minimalprogramm unmittelbar einlesen. Sie besteht aus Zeilen der Eingabe mit Kennzeichen (BIS-2.2.2 bis BIS-2.2.5, BIS-2.2.11): Kennzeichen K_1 K_2, Adreßteil, Operationsteil, Schlußpunkt, zum Beispiel `01 0040 000.` oder `20 0000 0053.`; die Hauptleitadresse steht als `21 …`-Zeile in der Datei oder wird vorher von Hand eingegeben. Das Minimalprogramm führt dabei die Adressenänderung nach K_2 aus; ein relativ adressiertes Programm kann deshalb auf eine beliebige Hauptleitadresse geladen werden. Voraussetzung ist, dass das Minimalprogramm selbst bereits auf der Trommel steht.

Weg 2: bitweise über das KEP. Das Minimalprogramm steht auf einer leeren Trommel nicht zur Verfügung; es wird selbst erst mit Hilfe des KEP geladen (BIS-2.1). Das KEP ist deshalb die Voraussetzung des BIS. Anmerkung (Festlegung des Projektinhabers): Es ist kein Bestandteil des BIS, aber dessen Voraussetzung. Weil das KEP weder Eingabekennzeichen noch Adressenänderung kennt (BIS-2.1), gilt für ein Programm, das über das KEP geladen werden soll:

1. **Das Programm muss absolut adressiert sein.** Jede Adresse ist die endgültige Trommeladresse. Im Assemblerquelltext heißt das: `start` ohne Operand und keine relativen Adressen, die erst beim Laden umgerechnet werden. Das Beispiel ist das Minimalprogramm selbst: Es ist absolut adressiert und wird mit der Hauptleitadresse 0 geladen.
2. **Das Programm besteht aus Leitbefehlen und aus abzuspeichernden Programmworten.** Der Leitbefehl bestimmt, wohin die folgenden Worte gehören (ein Transportbefehl a_1 a_2 a_3 a_4 `240` für die fortlaufende Abspeicherung ab der Zelle a_1 a_2 a_3 a_4, BIS-2.1). Ein neuer Leitbefehl ist nötig, wenn die Adressen nicht mehr fortlaufend sind. In der Quelldatei stehen Leitbefehle und Programmworte in Oktalschreibweise.
3. **Das KEP selbst liest nur Dualziffern.** Jede Oktalziffer wird deshalb durch ihre drei Bits ersetzt; geschrieben wird ab dem ersten von Null verschiedenen Bit. Der Leitbefehl endet mit „:“, jedes Programmwort mit „.“; ein Lochbandschlußzeichen gibt es bei der bitweisen Eingabe nicht (BIS-2.2.12.3). Beispiele:
   - Leitbefehl `0040 240` (Transport nach Zelle 0040), oktal `0040240`: Ziffernfolge `100 000 010 100 000` und das Schlußzeichen, also `100000010100000:`.
   - Programmwort `0001 164` (Befehl 164 mit Adreßteil 0001), oktal `00000001164`: `1001110100.`.

   Anmerkung (Festlegung des Projektinhabers): Diese Umsetzung nimmt der Assembler `c82ass` mit dem Schalter `--ABSPR` selbst vor. Er gibt das absolut adressierte Programm als Lochstreifen im binären R 300-Code aus, so wie das KEP ihn lesen kann (Leitbefehl am Anfang, dann die Worte; Dateiendung `.kep`), siehe BIS-2.2.11b.
4. **Ablauf:** KEP anspringen (BIS-2.2.12.2: `H G M↓ E E M↑`, Kanal anwählen, `R`), den Leitbefehl eingeben (`…:`), bei Eingabe über einen Lochbandleser auf „Halt“ schalten und den Kanal anwählen (BIS-2.2.12.3), dann die Worte (`….`) in der Reihenfolge der Adressen.

Anmerkung (Folgerung, nicht aus der Quelle): Eine Objektdatei im Kennzeichenformat (Weg 1) ist für das KEP unbrauchbar, weil die Kennzeichenzeilen aus Ziffern, Leerzeichen und Punkten bestehen, die das KEP als Dualziffern, Zwischenraum und Wortende liest. Aus einer solchen Datei entstünden falsche Worte.

Anmerkung (Folgerung, nicht aus der Quelle): Für Weg 1 gibt der Assembler `c82ass` die `.obj`-Datei mit dem Schalter `-o` in einen beliebigen Ordner aus. Der Aufruf im Ordner `Assembler/`, mit dem das Minimalprogramm in den Ordner des Lesers 1 ausgegeben wird, ist:

```
$ ./c82ass.sh -p Projekt_2 -s pj2-bis_minimalprogramm_v6.asm -o ../virtuelle_Geraete/Leser1
```

(`make_c82ass.sh` erzeugt dagegen nur das Programm `c82ass`, es assembliert nicht.) Ein Lochstreifenleser des Interpreters verlangt je Zeile ein Zeichen als 8 Ziffern 0/1 (virtueller Lochstreifen); eine Textdatei im Leserordner wird abgelehnt. Eine Textdatei, auch die `.obj`-Datei oder eine Datei mit bitweise notierten Worten, muss deshalb vorher mit `c8205_lochstreifen_umwandlung.py` in einen Lochstreifen umgewandelt werden. Die `.kep`-Datei von Weg 2 (Schalter `--ABSPR`) ist dagegen schon ein fertiger Lochstreifen und braucht keine Umwandlung.

Anmerkung (geprüft 22.10.2026): Ein Programm, das aus der oktalen Quelle die bitweise Folge für das KEP erzeugt, gibt es: den Assembler mit dem Schalter `--ABSPR` (BIS-2.2.11b); die `.kep`-Datei enthält keine Zeilenenden. Eine von Hand geschriebene Textdatei mit bitweisen Worten wird mit `c8205_lochstreifen_umwandlung.py` in einen Lochstreifen umgewandelt und erhält dabei für jedes Zeilenende das Zeichen SatzMarke/WRZL. Das KEP überliest dieses Zeichen zwischen den Worten. Begründung aus dem Code: Der Eingabe-Halt in Zelle 0030 (`0020 010`) spricht die Zelle 0020 ∨ Eingabe-Adresse an. WRZL hat die Eingabe-Adresse 0032 und ZWR die Eingabe-Adresse 0012; beide ergeben die Zelle 0032 (`0030 001`: Sprung nach 0030 mit Löschen von ⟨AC⟩). WRZL wirkt im KEP also wie ZWR. Zwischen zwei Worten ist ⟨AC⟩ nach dem Ausführen des Leitbefehls schon 0, das Zeichen bleibt ohne Wirkung. **Mitten in einem Wort** würde WRZL wie ZWR die bisher gelesenen Bits verwerfen (BIS-2.2.1a, Beispiel „111 ZR 101“). Prüfung: `Tests/bsp_5_18_kep_wrzl.test` liest denselben Streifen wie `bsp_5_18_kep.test`, mit je einer Zeile WRZL nach jedem : und . (Tastenfolge `H SW=1 G R`, weil die Pult-Tastatur WRZL nicht eingeben kann). Ergebnis: die Zellen 3000 bis 3004, 7723 und 7724 sind gleich wie ohne WRZL. Nicht geprüft: WRZL mitten im Wort, SatzMarke an einem echten Leser, die Zeichen WR und ZV (beide Eingabe-Adresse zur Zelle 0072, außerhalb des KEP).

BIS-2.2.11b Gesamtablauf: das KEP auf die Trommel bringen und ein Programm über das KEP laden (Ergänzung 21.10.2026)

Anmerkung (Festlegung des Projektinhabers): Das KEP ist die Wurzel allen Programmladens. Auf einer leeren Trommel gibt es nichts, womit sich das KEP selbst laden ließe; es muss deshalb mit Hilfsmitteln auf die Trommel gebracht werden, zum Beispiel mit dem TEST-Interpreter (Schalter `--LOADONLY`) oder durch Bearbeiten der Trommeldatei von Hand. Erst danach kann das KEP über den Lochstreifenleser das Minimalprogramm oder ein anderes absolut adressiertes Programm laden. Der folgende Ablauf zeigt alle Schritte mit den Aufrufen des Projekts. Die Aufrufe im Ordner `Assembler/` verwenden `c82ass.sh` (`make_c82ass.sh` baut nur den Assembler); Zielordner werden relativ zu `Assembler/` angegeben.

Schritt 1: das KEP assemblieren. Der Quelltext des KEP (`bsp_5_18_kep.asm`) ist absolut adressiert und beginnt auf Zelle 0000. Er wird mit dem Schalter `-t` zu einer Testdatei übersetzt, die der TEST-Interpreter laden kann:

```
$ cd Assembler
$ ./c82ass.sh -p Claude_bsp_reassemblierung/bsp_5_18_kep -s bsp_5_18_kep.asm -t ../Tests
C8205 Assembler C82aas Version 1.00-027
Projektverzeichnis: Claude_bsp_reassemblierung/bsp_5_18_kep
Benutze für -s die Datei: bsp_5_18_kep.asm

Pass 0
Verarbeite 'c82ass.makros', Erzeuge Macroersetzungen

Pass 1
Löse Literale auf

Pass 2
Löse Symbole auf nach Claude_bsp_reassemblierung/bsp_5_18_kep/bsp_5_18_kep.log und
    Claude_bsp_reassemblierung/bsp_5_18_kep/bsp_5_18_kep.sym
Startadresse: 0000

Pass 3
Erzeuge Kennzeichen-Befehle in Objektdatei Claude_bsp_reassemblierung/bsp_5_18_kep/bsp_5_18_kep.obj

0000: 01 0002 000    m0000: 000 m0002        ; Sprung zum Programmanfang des KEP
      20 0000 0002          equ 00002
0002: 01 0022 000    m0002: 000 m0022        ; Programmanfang
      20 0000 0014          equ 00014
0014: 00 7723 211    m0014: 211 m7723        ; <AC>:=<7723>
0015: 01 0025 210           210 m0025        ; <AC>:=<AC>+1°
0016: 00 7723 240    m0016: 240 m7723        ; <7723>:=<AC> = Leitbefehl
0017: 01 0030 001           001 m0030        ; <AC>:=0, Sprung zur Eingabe
0020: 01 0000 000           000 m0000        ; "0" Konstante im R 300-Code
0021: 00 40000000000        dw  040000000000 ; "1" Konstante im R 300-Code
0022: 01 0037 211    m0022: 211 m0037        ; <AC>:=0014 000
0023: 00 7724 240           240 d7724        ; <7724>:=0014 000
0024: 01 0030 001           001 m0030        ; "ZWR" im FS-Code, <AC>:=0
0025: 00 0001 000    m0025: 000 m0001        ; Konstante (Adresseneinheit 1°)
0026: 01 0000 000           000 m0000        ; "0" Konstante im FS-Code
0027: 00 40000000000        dw  040000000000 ; "1" Konstante im FS-Code (Zeile im Listing verdruckt, ergaenzt)
0030: 01 0020 010    m0030: 010 00020        ; E (0|1|:|.|ZWR), Eingabe-Halt
0031: 01 0030 006           006 m0030        ; <AC>:=Vzl(<AC>), Sprung zur Eingabe
0032: 01 0030 001           001 m0030        ; "ZWR" im R 300-Code, <AC>:=0
0033: 00 7723 000           000 m7723        ; "." im R 300-Code, Sprung zur Ausfuehrung des Leitbefehls
0034: 00 7723 000           000 m7723        ; "." im FS-Code
0035: 01 0016 000           000 m0016        ; ":" im R 300-Code, Leitbefehl speichern
0036: 01 0016 000           000 m0016        ; ":" im FS-Code
0037: 01 0014 000    m0037: 000 m0014        ; Konstante 0014 000

Ausgabe als Minimalprogramm-Objektdatei nach: Claude_bsp_reassemblierung/bsp_5_18_kep/bsp_5_18_kep.obj
Ausgabe der formatierten Speicherbelegung nach: Claude_bsp_reassemblierung/bsp_5_18_kep/bsp_5_18_kep.dmp
Ausgabe der Testdatei (Spalten 1,3,4,5 fuer K1=00/01) nach: ../Tests/bsp_5_18_kep.test
```

Schritt 2: das KEP auf die Trommel laden. Der TEST-Interpreter überträgt das KEP aus der Testdatei auf die Trommel und führt den Test nicht aus (`--LOADONLY`). Die Testdatei trägt `#TROMMEL=LEER`: Vorher wird die Trommel geleert, auf ihr steht danach nur das KEP (Zellen 0000, 0002 und 0014 bis 0037; Zelle 0001 und alles Übrige ist 0, also auch kein Rest eines früher geladenen Minimalprogramms). Im Projektordner:

```
$ cd ..
$ python3 c8205_interp.py --MODUS=TEST Tests/bsp_5_18_kep.test --LOADONLY

=== Test bsp_5_18_kep (#PROT STUFE=2) ===
--LOADONLY: Programm aus Tests/bsp_5_18_kep.test auf die Trommel <Pfad>/Trommel/trommel.txt geladen; Test nicht
    ausgefuehrt
```

Das KEP steht nun auf der Spur 0 der Trommel (Adressen 0000 bis 0037). Das Ersetzen dieser Schritte 1 und 2 durch das Bearbeiten der Trommeldatei von Hand ist ebenso möglich.

Schritt 3: das zu ladende Programm als KEP-Lochstreifen assemblieren. Das Minimalprogramm oder ein anderes, absolut adressiertes Programm wird mit dem Schalter `--ABSPR` in den Ordner des Lesers 1 ausgegeben. Es entsteht ein absolut adressierter Lochstreifen im binären R 300-Code, so wie ihn das KEP lesen kann (Dateiendung `.kep`; Aufbau siehe BIS-2.2.11a und die Assemblerdokumentation):

```
$ cd Assembler
$ ./c82ass.sh -p Projekt_2 -s pj2-bis_minimalprogramm_v6.asm -o ../virtuelle_Geraete/Leser1 --ABSPR
$ ./c82ass.sh -p Projekt_1/namen_raten -s namen_raten.asm -o ../virtuelle_Geraete/Leser1 --ABSPR
```

Der erste Aufruf ist der Regelfall: Das Minimalprogramm ist der Grundbaustein des BIS und kommt als erstes über das KEP auf die Trommel. Dafür muss `pj2-bis_minimalprogramm_v6.asm` vor `start` die Zeile `;#; LEITADRESSE 00` enthalten (Quelltext ist absolut geschrieben, siehe unten). Fehlt sie, läuft der Aufruf ohne Fehlermeldung durch, der Lochstreifen enthält aber bei 92 der 210 Zeilen des `.dmp` (v6 hat 218 Worte) falsche Adressen. Die Zeilenzahl der `.kep` unterscheidet sich mit und ohne Pragma, denn bei Kennzeichen 00 behält `--ABSPR` die führenden Nullbits der Wörter, bei Kennzeichen 01 lässt es sie weg (für v6 ist keine Kontrollzahl angegeben; sie ist keine Zahl von Zellen). Der zweite Aufruf (`namen_raten`) ist ebenfalls absolut geschrieben und trägt dasselbe Pragma mit `start` ohne Operand: Das Programm steht quasi an der Stelle des späteren Minimalprogramms und trägt in Zelle 0000 den Sprung zu seiner Startadresse (hier 02000) ein, genau wie das Minimalprogramm. Mit `start 02000` (ohne oder mit Pragma) fehlte Zelle 0000 im Lochstreifen (552 statt 553 Worte). Beim Minimalprogramm v6 gibt es keine Sprungzelle 0000: Sein Lochstreifen setzt nur Zelle 0001 mit dem Sprung zum Druck von BZ und AC (nach 0223); der KEP-Sprung in Zelle 0000 (nach 0002) und das KEP auf 0002 bis 0037 bleiben unberührt. Der Assembler warnt deshalb nur bei Wörtern auf den Zellen 0002 bis 0037. Bei Pragma `LEITADRESSE 00` gibt er einen Hinweis aus („Der Quelltext gilt als absolut adressiert, es wird keine Adresse umgerechnet“), keine Warnung. Der zweite zeigt, dass auf demselben Weg auch jedes andere absolut adressierte Programm geladen werden kann. Liegen mehrere Dateien im Ordner des Lesers 1, rückt die nach Änderungszeit, bei Gleichstand nach Dateiname, nächste nach; für das Laden sollte deshalb nur die gewünschte Datei dort liegen.

Adressen und Pragma (geprüft 22.10.2026): `--ABSPR` addiert zu jedem Wort mit Kennzeichen 01 die Ladeadresse (die Startadresse, nach jeder `equ`-Zeile die dortige Adresse). Zwei Schreibweisen führen zu einem richtig adressierten Lochstreifen:

- **Relativ geschriebener Quelltext** (Adressen ab 0000, ohne Zelle 0000 als Sprungzelle): **kein** Pragma, `start <adresse>`. Das Programm wird auf `<adresse>` geladen, die Operanden werden um `<adresse>` erhöht. Für ein Programm, das über das KEP seine Zelle 0000 setzen soll, ist diese Schreibweise nicht geeignet (Zelle 0000 fiele aus dem Streifen).
- **Absolut geschriebener Quelltext** (Adressen stehen schon fest, z. B. das Minimalprogramm und `namen_raten`): Pragma `;#; LEITADRESSE 00` vor `start`, `start` ohne Operand. Fehlt das Pragma, wird die Blockbasis zu den schon absoluten Adressen noch einmal addiert (beim Minimalprogramm 92 der 218 Worte falsch; v6 hat 218 Worte). Pragma 00 zusammen mit `start <adresse>` ungleich 0 ist falsch gemischt; der Assembler warnt (ab 1.00-028). Ein Wort auf einer Zelle 0002 bis 0037 würde das KEP überschreiben und wird gemeldet (ab 1.00-029); die Sprungzellen 0000 und 0001 sind erlaubt.

Das Laden selbst wurde mit dem Minimalprogramm geprüft: Nach dem Ansprung des KEP und dem Einlesen der `.kep` stimmen alle 218 Worte der Trommel mit der `.test`-Datei des Assemblers überein (nachgestellt am 24.10.2026 für v6 mit `testscripte/kep_laden_v6.sh`; Testdatei `testscripte/kep_laden_v6.test`, Streifen `Test-virtuelleGeraete/Leser1_02/pj2-bis_minimalprogramm_v6.kep`). Der Lauf endet nach dem Streifenende nicht von selbst (der Leser wartet auf eine weitere Datei); das Skript vergleicht deshalb die Trommel und beendet ihn. Die frühere Angabe „293 Worte“ galt der Vorgängerfassung.

Auch `namen_raten` wurde über das KEP geladen und mit `H G R` ausgeführt; der Projektinhaber bestätigt, dass es einwandfrei läuft (22.10.2026, von ihm selbst am Gerät-Modell ausgeführt, hier nicht nachgestellt).

Schritt 4: den GRAFIK-Interpreter starten.

```
$ cd ..
$ ./c8205_grafik.sh --HWCHECK
```

Der Interpreter findet auf Spur 0 (0000 bis 0037) das KEP vor. Es wird am Pult über den Speicherplatz 0002 angesprungen:

```
H  G  M+  E  E  M-  K1  R
```

(`M+` = M gedrückt halten, `M-` = M loslassen, `K1` = Kanal 1, Leser 1 anwählen; BIS-2.2.12.2).

Schritt 5: das KEP liest den Lochstreifen. Auf Kanal 1 liegt der binär codierte Lochstreifen mit dem Leitbefehl am Anfang (… `240:`) und den Programmworten (….). Das KEP liest den Leitbefehl, speichert ihn in der Zelle 7723 und überträgt jedes folgende Wort in die Zelle, auf die der Leitbefehl zeigt; nach jedem Wort wird die Adresse des Leitbefehls um eine Einheit erhöht. Ein Lochbandschlußzeichen gibt es nicht (BIS-2.2.12.3). Nach dem Lesen des Lochstreifens steht das Programm auf der Trommel; war es das Minimalprogramm, kann damit nun mit `H G R` die Eingabe mit Kennzeichen angesprungen werden, und weitere Programme lassen sich als Objektdatei laden (Weg 1 in BIS-2.2.11a).

BIS-2.2.12 Unterstützung elementarer Tastenkombinationen

Das Minimalprogramm ermöglicht es, mit wenigen Tastenkombinationen elementare Arbeiten am Programm auszuführen: Zellinhalte ausdrucken, fehlerhafte Plätze korrigieren, Befehlszähler und Akkumulator ausdrucken und das Kleine Eingabeprogramm ansprechen. Die Quelle schreibt die Tastenfolgen als Buchstabenfolgen (z. B. HGR); „H“ und „R“ stehen für die Stellungen des Schalters „H–R“.

Anmerkung (Lesehilfe, nicht Wortlaut der Quelle): In den Tastenfolgen mit M bedeutet der Pfeil ↓ „M gedrückt halten“ und ↑ „M loslassen“ (so auch in der Tastenliste zum Ansprung des KEP und in der Dokumentation des Interpreters). Die Tastenfolgen führen auf feste Einsprungzellen: `H G R` auf Zelle 0000 (Eingabe mit Kennzeichen), `H G M↓ E M↑ R` auf Zelle 0001 (Druck von Befehlszähler und Akkumulator; dort steht ein unbedingter Sprung zum Druckprogramm) und `H G M↓ E E M↑ R` auf Zelle 0002 (KEP). Allgemein bewirkt `H G M↓ E … E M↑ R` mit n-mal E den Sprung nach Speicherplatz n (n = 0 ohne M und E).

BIS-2.2.12.1 Druck des Befehlszählers und Akkumulators

Nach Programmunterbrechungen (Überlauf, Stopp usw.) ist es von Vorteil, den Befehlszählerstand und den Akkumulatorinhalt zu kennen. Da der C 8205 hierfür keine direkte Anzeigemöglichkeit besitzt, ist im Minimalprogramm ein entsprechendes Druckprogramm enthalten.

Nach dem Ausschreiben des Befehlszählerstandes und Akkumulatorinhaltes kann die Abarbeitung des Hauptprogrammes fortgesetzt werden (mit Ausnahme von Unterbrechungen in den Programmen „Adressendruck“, „Befehlsdruck“ und „Triadendruck“).

Vorgehen:

1. Im ersten Schritt nach der Programmunterbrechung wird mit Hilfe von HGÜE der Akkumulatorinhalt in die Zelle 7777 transportiert (H: Schalter H–R auf H; G: Generallöschen; Ü: Übertragstaste, stellt `7777 240` auf; E: Einzelschritt, führt den Befehl aus; Tasten s. Bedienpult).
2. Durch die Tastenkombination `H G M↓ E M↑ „SP Lösen“ R` ist anschließend das Druckprogramm anzuspringen. Vor dem Drücken von „R“ ist der Schalter „SP“ (einrastender Selektorschalter für den bedingten Sprung mit Rückkehrabsicht, s. Bedienpult) zu lösen.
3. Ausgegeben werden die Adresse des zuletzt abgearbeiteten Befehls und der zugehörige Akkumulatorinhalt in Triadendarstellung.
4. Danach wird die Abarbeitung des Programms fortgesetzt, oder der Automat geht in den Zustand „bedingter Stopp“, wenn der Schalter „BS“ eingeschaltet ist.

War der letzte im Hauptprogramm abgearbeitete Befehl ein Ausgabebefehl, so wird dieser bei Fortsetzung des Programms noch einmal ausgeführt.

Anmerkung (Lesehilfe, nicht Wortlaut der Quelle): Die Tastenfolge mit einmaligem E ist der Sprung nach Speicherplatz 1; die Ansprungzelle dieses Hilfsprogramms ist also 0001. Danach steht im Akkumulator die um 2 Einheiten erhöhte Adresse des nächsten Befehls.

BIS-2.2.12.2 Ansprung des Kleinen Eingabeprogramms

Das Kleine Eingabeprogramm (KEP) kann auf dem Speicherplatz 0002 angesprungen werden. Nach dem Ansprung ist manuell der Eingabekanal anzuwählen. Die Eingabe kann über die Tastatur, die beiden Lochbandleser und die Schreibmaschine erfolgen.

Beim Ansprung des KEP sind folgende Tasten zu drücken:

Schritt	Bedienung	Handhabung
(a)	Schalter „H–R“	H drücken
(b)	Taste G	drücken
(c)	Taste M	gedrückt lassen
(d)	Taste E	zweimal anschlagen
(e)	Taste M	loslassen
(f)	K0, K1, K2	anwählen
(g)	Schalter „H–R“	R drücken

Danach geht der Automat in den Zustand „Eingabe“ und ist zum bitweisen Einlesen über den angewählten Kanal bereit.

Anmerkung (Lesehilfe, nicht Wortlaut der Quelle): Das zweimalige E unter gehaltenem M ist der Sprung nach Speicherplatz 2. Auf 0002 beginnt das eigentliche KEP.

BIS-2.2.12.3 Eingabe eines bitweise gelochten Programms

Beispiel der Quelle: Das Lochband für den Baustein B 1.1 (Minimalprogramm), die Konstanteneingabespur sowie das Ansprungprogramm für die B- und D-Programme, das bitweise gelocht ist, soll mittels KEP über Leser 1 eingegeben werden. Die angegebenen Programme belegen die Speicherplätze 40–437. Es wird angenommen, daß der Leitbefehl nicht auf dem Lochband steht. (In der Praxis ist der Leitbefehl auf dem Lochband enthalten.)

1. **Ansprung des KEP:** `H G M↓ E E M↑ (T) R`
2. **Aufbau des Abspeicherbefehls** `40 240` **als Leitbefehl** (Transport nach Zelle 40): Die interne Darstellung des Befehls ist `000 … … 000 100 000 010 100 000`. Beginnend mit dem ersten von Null verschiedenen Bit ist daher die Zeichenfolge `100 000 010 100 000 :` einzugeben. Durch das Schlußzeichen „:“ wird das im Akkumulator aufgebaute Wort zum Leitbefehl erklärt und in einer festen Zelle (7723) gespeichert.
3. **Eingabe über Leser 1:** Nach Eingabe des Doppelpunktes ist, um Kanal 1 anwählen zu können, auf „Halt“ zu schalten. Nach dem Drücken von K1 ist der Rechner bereit, das erste Wort über Leser 1 einzulesen. Nach dem Aufbau des Wortes im Akkumulator wird durch das Schlußzeichen Punkt „.“ der Leitbefehl `40 240` angesprungen. Er bewirkt, daß das Wort nach der Zelle 40 transportiert wird. Durch die anschließende Erhöhung der Adresse des Leitbefehls um eine Einheit wird die Abspeicherung des nächsten Wortes vorbereitet.

Bei der bitweisen Eingabe gibt es kein Lochbandschlußzeichen.

BIS-3 Abgrenzung zu Interpreter und Assembler

Der Interpreter kennt das BIS nicht: Das BIS ist Anwendungs-Software auf dem C 8205 und kein Teil der simulierten Hardware. Der Assembler kennt das BIS (er erzeugt Objektcode für die Eingabe mit Kennzeichen über das Minimalprogramm und mit `--ABSPR` den binären Lochstreifen für das KEP), ist aber ein eigenes Teilprojekt. Anmerkung (Folgerung, nicht aus der Quelle): Es ließe sich als Programm, etwa als Trommeldatei eines Tests, im Interpreter ausführen.

BIS-A Anhang: Ansprungzellen, Anschlußbedingungen und Arbeitszellen

Das Prinzip der Anschluß- und Ansprungzellen steht in BIS-1.2; die folgenden Tabellen legen die Adressen fest.

BIS-A.1 Eingabe- und Hilfsprogramme (Anschlußzellen 201 bis 430)

Erläuterung der Quelle (Wortlaut sinngemäß, Seitenabbild „Anschluß- und Ansprungzellen"): Das Minimalprogramm verarbeitet die Kennzeichen $K_1 = 3$ bis 9 nicht selbst, sondern enthält nur die Anschlußbedingungen für die entsprechenden Leseprogramme. Wird z. B. $K_1 = 5$ eingegeben, muss außer dem Minimalprogramm noch der Baustein B 2.2 abgespeichert sein. $K_1 = 5$ bewirkt dann einen Sprung zu dem in B 2.2 enthaltenen Eingabeprogramm für Dezimalzahlen. Nach dem Einlesen der Zahl erfolgt die Rückkehr zum Minimalprogramm; dieses speichert die Zahl ab und geht über zur Eingabe eines neuen ersten Kennzeichens.

Für die Herstellung der Anschlußbedingungen im Minimalprogramm ist notwendig, dass beim Einlesen eines Eingabebausteins B 2.1, B 2.2 oder B 2.3 bestimmte Zellen im Minimalprogramm versorgt werden. Dem Minimalprogramm muss also mitgeteilt werden, wohin der jeweilige Baustein abgespeichert wird. Bei der 8-Kanal-Variante des Minimalprogramms werden Speicherplätze der Spur 200–240 als Anschlußzellen verwendet. Den Kennzeichen K_1 entsprechen im Einzelnen die Zellen der folgenden Tabelle.

Anmerkung (Festlegung des Projektinhabers): $K_1 = D$ bezeichnet die Taste D der Bedienpult-Tastatur, nicht den Buchstaben D des Schreibwerks.

Baustein	Programm	Anschlußzelle	Anschlußbedingung
B 2.1	Eingabe einer Dezimalzahl (Umwandlung in Gleitkommazahl) mit $K_1 = 3$	203	02 4000 0253 000.
B 2.1	Eingabe einer Gleitkommazahl mit $K_1 = 4$	204	02 4000 0213 000.
B 2.2	Eingabe einer Dezimalzahl (Umwandlung in Festkommazahl, K. h.) mit $K_1 = 5$	205	02 4000 0114 000.
B 2.3	Eingabe einer Dezimalzahl mit maximal 8 Ziffern (Umwandlung in Tetraden) mit $K_1 = 6$	206	02 4000 0060 000.

C 8205 – Bausteininterpretationssystem

Quelle: C8205-Interpreter-BIS.md · im Wortlaut der Quelle

Baustein	Programm	Anschlußzelle	Anschlußbedingung
B 2.3	Eingabe einer Dezimalzahl mit maximal 8 Ziffern (Umwandlung in Festkommazahl, K. h.) mit $K_1 = 7$	207	02 4000 0062 000.
B 2.1	Eingabe von Dezimalzahlen (Umwandlung in Gleitkommazahlen) ohne Kennzeichen im Satz mit $K_1 = 8$	210	02 4000 0167 000.
B 2.2	Eingabe von Dezimalzahlen (Umwandlung in Festkommazahlen, K. h.) ohne Kennzeichen im Satz mit $K_1 = 9$	211	02 4000 0102 000.
B 2.5	Eingabe von Externbefehlen mit $K_1 = +$	220	02 4000 0135 000.
–	Ansprung der Hilfsprogramme (Bedienungs- und Kontrollprogramme) mit $K_1 = D$ (Taste D der Bedienpult-Tastatur, nicht der Buchstabe D des Schreibwerks)	224	
B 6.2	Bereichsausschreiben mit D3	403	02 10 000.
B 6.2	Bereichsausschreiben mit D4	404	02 26 000.
B 6.2	Bereichsausschreiben mit D5	405	02 55 000.
B 6.1	Protokollprogramm mit D6	406	02 ... 000.
B 6.1	Protokollprogramm mit D7	407	02 144 000.
B 4.1	Bereichsausstanzen mit D8	410	02 ... 000.
B 6.3	Vergleichsprogramm mit B0	430	02 ... 000.

Anmerkung (24.10.2026, Lieferung 003): Der Bereich 0400 bis 0477 mit den Anschlusszellen 403 bis 410 und 430 ist im Minimalprogramm v6 frei; der Druck von BZ und AC liegt jetzt in 0040 bis 0377 (nicht mehr auf 0500 bis 0533) und druckt ohne die Bezeichnungen „BZ“ und „AC“ (Zeile `0000 bbbb 000 aaaa aaaa aaa`; Annahme, vom Inhaber noch zu bestätigen). **Offener Punkt (Beobachtung):** Die Anschlusszellen 203, 205 bis 207, 210, 211, 220 und 224 liegen im Bereich 0040 bis 0377 und sind im Quelltext von Befehlen oder Konstanten der Kennzeicheneingabe belegt; nur 204 ist frei. Ein Baustein, der seine Anschlussbedingung dort ablegt, würde diese Zellen überschreiben.

BIS-A.2 Rechen-, Ausgabe- und Satzeingabeprogramme (Ansprungzellen 7540 bis 7575)

Baustein	Programm	Ansprungzelle	Anschlußbedingung
B 5.5	Festkommadivision ohne Rückspeicherung	7540	02 217 000.
B 5.5	Festkommadivision mit Rückspeicherung	7541	02 73 000.
B 5.4	Festkommamultiplikation ohne Rückspeicherung	7542	02 132 000.
B 5.4	Festkommamultiplikation mit Rückspeicherung	7543	02 70 000.
B 5.3	Gleitkommadivision ohne Rückspeicherung	7544	02 134 000.
B 5.3	Gleitkommadivision mit Rückspeicherung	7545	02 134 000.
B 5.2	Gleitkommamultiplikation ohne Rückspeicherung	7546	02 34 000.
B 5.2	Gleitkommamultiplikation mit Rückspeicherung	7547	02 34 000.
B 5.1	Gleitkommasubtraktion ohne Rückspeicherung	7550	02 440 000.
B 5.1	Gleitkommasubtraktion mit Rückspeicherung	7551	02 56 000.
B 5.1	Gleitkommaaddition ohne Rückspeicherung	7552	02 140 000.
B 5.1	Gleitkommaaddition mit Rückspeicherung	7553	02 456 000.
B 4.5	Ausgabe von Tetraden in Dezimalschreibweise ohne Runden	7554	02 275 000.
B 4.5	Ausgabe von Festkommazahlen in Dezimalschreibweise ohne Runden (Variante II)	7555	00 27 210.
B 4.5	Ausgabe von Festkommazahlen in Dezimalschreibweise mit Runden (Variante II)	7556	02 14 000.
B 4.4	Ausgabe von Festkommazahlen in Dezimalschreibweise (Variante I)	7557	02 12 000.
B 4.3	Ausgabe einer Gleitkommazahl in Dezimalschreibweise ohne Runden	7560	00 27 210.
B 4.3	Ausgabe einer Gleitkommazahl in Dezimalschreibweise mit Runden	7561	02 156 000.
B 4.2	Ausgabe einer Gleitkommazahl in Gleitkommadarstellung	7562	02 100 000.
B 4.1	Stanzen eines Wortes in Triaden mit $K_1 = 0$	7563	02 26 000.
B 2.4	Eingabe von Dezimalzahlen mit Kommainformation (Umwandlung in Festkommazahlen, K. h.) im Satz ohne Kennzeichen	7564	02 45 000.
B 2.2	Eingabe von Dezimalzahlen (Umwandlung in Festkommazahlen, K. h.) im Satz ohne Kennzeichen	7565	02 43 000.
B 2.1	Eingabe von Dezimalzahlen (Umwandlung in Gleitkommazahlen) im Satz ohne Kennzeichen	7566	02 267 000.
B 2.1	Eingabe von Gleitkommazahlen im Satz ohne Kennzeichen	7567	02 270 000.
B 2.3	Eingabe einer Dezimalzahl mit maximal 8 Ziffern (Umwandlung in Tetraden) ohne Kennzeichen	7570	02 14 000.
B 2.3	Eingabe einer Dezimalzahl mit maximal 8 Ziffern (Umwandlung in Festkommazahl, K. h.) ohne Kennzeichen	7571	02 53 000.
B 2.4	Eingabe einer Dezimalzahl mit Kommainformation (Umwandlung in Festkommazahl, K. h.) ohne Kennzeichen	7572	02 53 000.
B 2.2	Eingabe einer Dezimalzahl (Umwandlung in Festkommazahl, K. h.) ohne Kennzeichen	7573	02 121 000.
B 2.1	Eingabe einer Dezimalzahl (Umwandlung in Gleitkommazahl) ohne Kennzeichen	7574	02 335 000.
B 2.1	Eingabe einer Gleitkommazahl ohne Kennzeichen	7575	02 336 000.

Anmerkung (Deutung, vom Projektinhaber am 07.11.2026 als sehr plausibel bezeichnet, nicht aus der Quelle belegt): Die Zellen 7555 und 7560 enthalten statt eines Sprungs den Maschinenbefehl `210 0027` (Anschlußbedingung `00 27 210.`). Er läuft in die Folgezelle 7556 bzw. 7561 weiter, die den Sprung der Ausgabe mit

Runden enthält. Für diese beiden Zellen gilt die Regel aus BIS-1.2 (Baustein trägt nur einen unbedingten Sprung ein) also nicht.

BIS-A.3 Arbeitszellen des Minimalprogramms

Arbeitszellen sind feste Zellen, die das Minimalprogramm als Speicher für Zwischenwerte und Zustände benutzt. Sie liegen im Arbeitsbereich 7600 bis 7777 (Definition unten). Sie sind zu unterscheiden von den Ansprungzellen (Tabellen oben, 7540 bis 7575), die nur dem Ansprung von Programmen dienen; die Spur der Ansprungzellen gehört den anderen Bausteinen, nicht dem Minimalprogramm. Aufgenommen werden nur Zellen, die das Minimalprogramm benötigt (Ausnahme: die Register R_2 und R_3, die zusammen mit R_1 genannt werden). Die Tabelle wird schrittweise aufgebaut und fortgeschrieben, sobald zu einer Zelle etwas geklärt ist. Alle Zahlen sind oktal.

Definition: Register R_1, R_2, R_3. Die Register R_1, R_2 und R_3 sind allgemeine Arbeitszellen. Sie werden von den Programmteilen des BIS unterschiedlich genutzt. Deshalb darf keinem Register eine feste Verwendung zugeordnet werden; die Tabelle unten nennt nur, wie ein Register im jetzigen Stand benutzt wird, nicht wofür es bestimmt ist. Die Zuordnung von R_1 als Versorgungszelle der Ausgabeunterprogramme (BIS-2.2.10) gilt nur als Schnittstelle für den Augenblick des Aufrufs: Der Aufrufer legt dort den auszugebenden Wert ab, das Unterprogramm verändert ihn nicht. (Festlegung des Projektinhabers)

Definition: Arbeitsbereich 7600 bis 7777. Für das BIS stehen die vier Spuren 7600, 7640, 7700 und 7740 als Arbeitszellen zur Verfügung (7600 … 7777, je Spur 32 Zellen, zusammen 128 Zellen). Darin liegen auch die oben genannten festen Zellen (Leitbefehl 7723 und Hauptleitadresse 7724 des KEP, Register, Transportzelle 7777); diese sind belegt. (Festlegung des Projektinhabers; die Quelle des Projektinhabers nennt „770“, gelesen als Spur 7700, da die vier Spuren den Bereich 7600 … 7777 lückenlos füllen.)

- **Verlegung in den Arbeitsbereich:** Arbeitszellen, die im Quelltext des Minimalprogramms mit `db` im Programmbereich angelegt sind (z. B. Zähler, Rückkehrbefehle, gesicherte Werte), werden stattdessen mit `abs` auf Zellen des Arbeitsbereichs gelegt. Das spart Platz im Kernprogramm. Ausgenommen sind Konstanten, die geladen werden müssen: Sie bleiben als `dw` im Programmbereich, weil ihr Inhalt beim Laden des Minimalprogramms mitgeschrieben wird.
- **Nutzung für Befehle mit BAZ B = 4, 5, 6, 7:** Die Zellen des Arbeitsbereichs dürfen auch als Zellenfeld der Gruppen- und Wiederholungsbefehle (G, GX, W, WX) dienen, sofern die einzelnen Zellen dabei nicht anderweitig in Benutzung sind. Diese Befehle laufen über aufeinanderfolgende Zellen und verändern oder lesen jede davon; eine belegte Zelle im Feld würde also zerstört bzw. falsch gelesen.
- **„In Benutzung“:** Eine Zelle gilt als in Benutzung, wenn irgendein Teil des gesamten BIS-Minimalprogramms sie belegt (also einschließlich KEP und aller Unterprogramme), nicht nur der Teil, der gerade abläuft. Maßgeblich für die Belegung ist die Tabelle unten bzw. der Quelltext `pj2-bis_minimalprogramm_v6.asm`. Eine Zelle, die kein Teil des Minimalprogramms belegt, ist frei.
- **Absolute Adressen:** Die Adressen der festen Zellen (die `abs`-Symbole des Quelltextes, Tabelle unten) sind absolut und werden nie verschoben.

Feste Zellen mit besonderer Bedeutung (Stand v6).

Zelle	Bezeichnung	Verwendung	Stand
7723	Leitbefehl des KEP	KEP: „:“ speichert den im Akkumulator aufgebauten Befehl hier (Zelle 0016); „.“ führt ihn aus (Zellen 0033 und 0034) und erhöht seine Adresse um eine Einheit (Zellen 0014 bis 0016). Beispiel: Leitbefehl `40 240` für die Abspeicherung ab Zelle 40. Das Minimalprogramm v6 benutzt diese Zelle nicht; sie liegt auf der Druckspur 7700 bis 7737 und wird vom Druck überschrieben (Tabelle unten).	für das KEP geklärt
7724	Hauptleitadresse des KEP	KEP: Zelle 0023 setzt hier den Anfangswert `0014 000` (aus Zelle 0037 über Zelle 0022), weil sie beim frisch geladenen KEP noch nicht gesetzt ist. **Das Minimalprogramm v6 setzt seine Hauptleitadresse nicht hier, sondern auf 7611 (`LA`+1).** Die Zelle 7724 dient dem Druck nur noch als Adresse der Spur 7700 bis 7737 (`adr7724`, Befehle 440).	geklärt (Quelltext v6)

Zelle	Bezeichnung	Verwendung	Stand
7777	Transportzelle für den Akkumulator / Übergabezelle	Ziel von HGÜE (aufgestellter Befehl `7777 240`, Akkumulatorinhalt wird nach 7777 transportiert). Quelle für die Ausgabe mit D0, D1, D2 und für das Druckprogramm von Befehlszähler und Akkumulator. In v6 ist 7777 die interne Arbeitszelle der Druckprogramme: Sie kopieren das Wort aus R_1 (7773) hierher und verschieben es dort (`adr7777`); die Übergabe vom Aufrufer erfolgt über R_1.	geklärt
7773	R_1 (Register 1)	Keine feste Verwendung (s. Definition oben). Im Assembler als `r1: abs 07773` definiert. **Das Minimalprogramm v6 benutzt R_1 als Übergabezelle** der Druckunterprogramme (Triadendruck, Adressdruck, Befehlsdruck; `adr7773`, BIS-2.2.10): Der Aufrufer legt das Wort in R_1 ab, das Druckprogramm kopiert es nach 7777 und arbeitet dort weiter; R_1 wird nie beschrieben. Der Druck von BZ und AC legt seine Wörter ebenfalls in R_1 ab.	geklärt (Angabe des Projektinhabers); Übergabe in v6 gleich BIS-2.2.10 (Stand 24.10.2026)
7774	R_3 (Register 3)	Keine feste Verwendung (s. Definition oben); häufig benutzte Arbeitszelle der Programme, spart Speicher im Programmbereich. Im Assembler als `r3: abs 07774`. Das Minimalprogramm benutzt R_3 nicht.	geklärt (Angabe des Projektinhabers)
7776	R_2 (Register 2)	Keine feste Verwendung (s. Definition oben); häufig benutzte Arbeitszelle der Programme, spart Speicher im Programmbereich. Im Assembler als `r2: abs 07776`. Das Minimalprogramm benutzt R_2 nicht.	geklärt (Angabe des Projektinhabers)

Belegung des Arbeitsbereichs 7600 bis 7777 im jetzigen Stand des Minimalprogramms (Quelltext `pj2-bis_minimalprogramm_v6.asm`, geprüft gegen die `abs`-Symbole und den Code am 24.10.2026). Die Tabelle führt alle Zellen des Bereichs, die das gesamte Minimalprogramm belegt. Die Zellen 7773 und 7777 sind in der Tabelle oben beschrieben und hier nur der Vollständigkeit halber aufgeführt.

Zelle	Name	Verwendung
7600	`LB`	Leitbefehl (Abspeicherbefehl `240` mit Adresse): `LB` := (7611) + `RA` + 0240; wird bei jedem Einstieg und nach jeder Änderung neu gebaut (`lb_neu`)
7601	`LB1`	Folgezelle des Leitbefehls: Sprung zurück (`kz_sp` setzt hier den Befehl `000 kz_gesp`)
7602	`XW`	Arbeitszelle für sofort auszuführende Befehle ($K_1 = 1$, sowie die Befehle zum Speichern einer Leitadresse $K_1 = 2$)
7603	`XW1`	Folgezelle von `XW`: Stoppbefehl `020` ($K_1 = 1$) bzw. Sprung zum neuen Kennzeichen ($K_1 = 2$)
7604	`XW2`	Folgezelle von `XW1`: Sprung zum neuen Kennzeichen (`kz_neu`) nach dem Stopp von $K_1 = 1$
7610	`LA`	Leitadresse 0, immer 0 (`kz_anf` setzt sie auf 0); Basis der Adressrechnung
7611 bis 7621	`LA`+1 bis `LA`+9	Leitadressen 1 bis 9 (x-te Leitadresse auf `LA` + x, also 7611 bis 7617, 7620 und 7621; $K_2 = 1$ bis 9; Wert = Adresse · 01000). `LA`+1 = 7611 ist die Hauptleitadresse des Minimalprogramms: gesetzt mit $K_1 = 2$, $K_2 = 1$ (dabei `RA` := 0), Ablageort und Basis der Adressänderung $K_2 = 1$. Gelesen wird mit dem R-Lesebefehl 251
7630	`kz_rs`	Kennzeicheneingabe: Rückkehrbefehl zum Aufrufer
7631	`kz_k1`	Kennzeicheneingabe: K_1 (Wert der Ziffer, 0 bis 15)
7632	`kz_k2`	Kennzeicheneingabe: K_2
7633	`kz_k2s`	Kennzeicheneingabe: Wert der K_2-ten Leitadresse (bei $K_2 = 0$ gleich 0)
7634	`kz_a`	Kennzeicheneingabe: akkumulierte Zahl (Wort bzw. Adresse)
7635	`kz_d`	Kennzeicheneingabe: zuletzt gelesene Ziffer
7636	`kz_z`	Kennzeicheneingabe: Ziffernzähler (Einsen, je Ziffer um 1 Bit nach links; zehn Einsen für elf Ziffern, sieben Einsen für die acht Ziffern der Adresse)
7640 bis 7677	`nullspur`	Spur mit ausschließlich Nullen, für Anwenderprogramme gesperrt. `kz_anf`, `ae_anf` und die Druckprogramme löschen sie (441). Auf ihr liegen die Nullzellenketten des Wiederholungsbefehls 616: `nz_12` auf 7644 (12 Linksverschiebungen), `nz_9` auf 7653 (9) und `nz_3` auf 7671 (3)

Zelle	Name	Verwendung
7700 bis 7737	Druckspur	Der Triadendruck, Adressdruck und Befehlsdruck legen mit 440 eine Spur von Einsen auf diese Spur (Adresse 7724, `adr7724`); die GX-Gruppenbefehle laufen über sie (Adresse 7721, `adr7721`). Sie wird bei jedem Druck überschrieben und kann nicht als Arbeitsspur dienen; auch 7723 und 7724 des KEP gehen dabei verloren. Darin liegen die Arbeitszellen 7717 (`td_g`, Gruppenzähler, kein Sektor des GX-Befehls) und 7720 (`td_z`, Zähler der noch zu druckenden Ziffern; Einsen, werden nach links geschoben), die nach dem Anlegen der Einsen gesetzt werden
7740	`ae_rs`	Adresseneingabe: Rückkehrbefehl zum Aufrufer
7741	`td_rs`	Triadendruck (und damit Adress- und Befehlsdruck): Rückkehrbefehl zum Aufrufer
7742	`RA`	Relativadresse, Wert = Adresse · 01000: nach jedem gespeicherten Wort + 01000; $K_1 = 2$ / $K_2 = 0$ setzt `RA` := Adresse, $K_2 = 1$ setzt `RA` := 0. Der Leitbefehl wird aus Hauptleitadresse + `RA` gebaut
7743	`dr_rs`	Druck von BZ und AC (Zelle 0001, Code ab 0223): Fortsetzungsbefehl `000 b`, aus dem um 3 Adresseinheiten verminderten AC gebildet; die Routine springt am Ende mit `000 dr_rs` dorthin
7744	`dr_ac`	Druck von BZ und AC: gesicherter AC des unterbrochenen Programms (7777 wird vom Druck überschrieben, deshalb zuerst gesichert)
7760	`td_z`	Triadendruck (und Adress-/Befehlsdruck): Ziffernzähler; liegt auf Spur 7740, weil die Einsen-Spur 7700 bis 7737 ihn überschriebe
7773	`adr7773`	R_1: Übergabezelle der Druckprogramme (Wort vom Aufrufer, bleibt unverändert; s. Tabelle oben)
7777	`adr7777`	Transportzelle des Akkumulators; zugleich Arbeitszelle der Druckprogramme (s. Tabelle oben)

Das sind 94 vom Minimalprogramm benutzte Zellen des Arbeitsbereichs (22 auf Spur 7600, 32 Nullspur, 32 Druckspur, 8 auf Spur 7740: 7740 bis 7744, 7760, 7773 und 7777). Sie stehen nicht im `.dmp`, denn dort erscheinen nur die Zellen des Programms (v6: 218 Worte auf 0001 und 0040 bis 0377).

Freie Zellen. Alle übrigen 34 Zellen des Arbeitsbereichs sind unbenutzt: 7605 bis 7607, 7622 bis 7627, 7637, 7745 bis 7757, 7761 bis 7772 und 7774 bis 7776. Die Zellen 7746, 7750, 7753, 7755, 7764 und 7771 stehen in der Liste der festen Zellen unten (nicht mit „Bereichsausschreiben“ ausschreibbar); ihre Verwendung ist noch zu klären, sie sind deshalb als Arbeitszellen mit Vorsicht zu wählen. Die Zellen der Druckspur 7700 bis 7737 sind nicht frei (s. o.), obwohl der Quelltext sie nur während des Drucks benutzt. Zu den Beispieltests, die 7700 bis 7737 als Arbeitsspur belegen, s. die Anmerkung (Änderung) unten.

Anmerkung (Folgerung aus der Verlegung, nicht aus der Quelle): Die Zelle `dr_rs` lag früher als `db` unmittelbar hinter dem bedingten Stopp (`060 0`) und wurde durch Weiterlaufen ausgeführt. Liegt eine solche Zelle im Arbeitsbereich, muss die Routine sie ausdrücklich anspringen (`000 dr_rs`). Der Platz im Programmbereich bleibt dabei gleich, weil der Sprung die frühere Zelle belegt; Verlegungen sparen nur dort Platz, wo die Zelle nicht durch Weiterlaufen ausgeführt wurde.

Zellen 7554 bis 7563 (Ansprungzellen der anderen Bausteine). Diese Zellen sind die Speicherplätze der Ansprungzellen der Ausgabe- und Eingabebausteine B 4.x, B 2.x und B 5.1 (BIS-2.2.10, BIS-A.2) und gehören nicht dem Minimalprogramm (Angabe des Projektinhabers). Das Minimalprogramm liest und beschreibt sie nicht. Seine Arbeitszellen der Ausgabe- und Eingabeunterprogramme (Triadendruck, Befehlsdruck, Adressendruck, Adresseneingabe) liegen im Arbeitsbereich (Tabelle oben). Ihre Adressen sind absolut und werden nie verschoben; Zähler und Rückkehrbefehle dort sparen Speicher im Programmbereich (bei Konstanten, die geladen werden müssen, geht das nicht). Im Assembler sind sie als `abs`-Symbole definiert.

Weitere feste Zellen, deren Verwendung noch zu klären ist: Die Quelle nennt die folgenden Zellen als nicht mit „Bereichsausschreiben“ ausschreibbar: 7645, 7647, 7657, 7660, 7661, 7705, 7706, 7710, 7711, 7713, 7714, 7722, 7723, 7725, 7730, 7731, 7737, 7746, 7750, 7753, 7755, 7764, 7771, 7773. Anmerkung: Die Liste enthält auch 7773; das ist das Register R_1. Anmerkung (Folgerung, nicht aus der Quelle): Da 7723 (Leitbefehl) in dieser Liste steht, sind die übrigen Zellen Kandidaten für Arbeitszellen des Minimalprogramms. Welche davon tatsächlich dazu gehören und wofür, ist noch zu klären. Stand im Quelltext v6: Von den Zellen dieser Liste belegt das Minimalprogramm 7645, 7647, 7657, 7660 und 7661 (Nullspur 7640 bis 7677) und überschreibt mit dem Druck 7705, 7706, 7710, 7711, 7713, 7714, 7722, 7723, 7725, 7730, 7731 und 7737 (Druckspur 7700 bis 7737). Frei sind 7746, 7750, 7753, 7755, 7764 und 7771; 7773 (R_1) ist die Übergabezelle der Druckprogramme.

BIS-A.4 Funktionen des Minimalprogramms v6 (Stand 24.10.2026)

Quelltext `Assembler/Projekt_2/pj2-bis_minimalprogramm_v6.asm`, Code in 0040 bis 0377 (218 Worte). Aufruf der Unterprogramme mit `025 <Ansprungadresse>`, Rückkehr zum Aufrufer. Es gibt kein Hauptprogramm. „Prüfskript“ nennt die Tests (`Tests/…test`, Soll in `Test-Ergebnisvergleiche/`) und das Prüfprogramm (`testscripte/pj2-bis_minimalprogramm_test.sh`: `druck`, `adresse`, `wort`, `progst`; Tests `Tests/v6_test_*.txt`).

Name	Ansprungadresse	Kurzbeschreibung	Parameterversorgung	Testscript
Druck BZ und AC (`druck`)	0001 (Sprung nach 0223)	druckt BZ- und AC-Wort des unterbrochenen Programms in einer Zeile und setzt es fort	AC des Programms in 7777 (`H G Ü E`), BZ am Pult; Tastenfolge `H G Ü E G M+ E M- R`; Kanal K2 wählt der Druck selbst	`pj2-bis_v6_druck_bz_ac`, `_druck_interrupt`, `_druck_interrupt_neu`
Triadendruck (`triadendruck`)	0200	11 Triaden des Wortes, in Gruppen zu 4 Ziffern	Wort in R1 (7773), Kanal K2 vorher angewählt	`v6_test_druck` (Treiber `Tests/v6_test_druck.txt`)
Adressdruck (`adressdruck`)	0220	mittlere 4 Triaden (Adressteil)	Wort in R1 (7773), Kanal K2 angewählt	`pj2-bis_v6_druck_adressdruck`
Befehlsdruck (`befehlsdruck`)	0240	untere 7 Triaden (Befehl)	Wort in R1 (7773), Kanal K2 angewählt	`pj2-bis_v6_druck_befehlsdruck`
Eingabe Wort mit Kennzeichen (`eingabewortkz`)	0262 (→ `kz_anf` 0040)	liest K1 K2 und Ziffern bis Schlusszeichen :/BM	Eingabekanal vorher angewählt (z. B. `H G K1 R`); Hauptleitadresse mit `2 1 aaaa .`, Relativadresse mit `2 .` bzw. `2 0 aaaa .` vom Band	`pj2-bis_v6_kz_*` (10 Tests), `v6_test_wort`, `v6_worte`, `v6_leitadressen`
Eingabe Programmstück (`eingabeprogst`)	0263 (→ `kz_anf` 0040)	wie 0262, Wörter fortlaufend ab Hauptleitadresse + RA	wie 0262	`pj2-bis_v6_ps_*` (13 Tests), `v6_test_progst`, `v6_programmstueck`
Adresseneingabe (`adresseneingabe`)	0316 (→ `ae_anf` 0044)	liest bis 8 Oktalziffern mit Punkt, Ergebnis im AC (t3 bis t10)	Eingabekanal angewählt; Rückkehr mit Adresse im AC	`pj2-bis_v6_ae_*` (6 Tests), `v6_test_adresse`